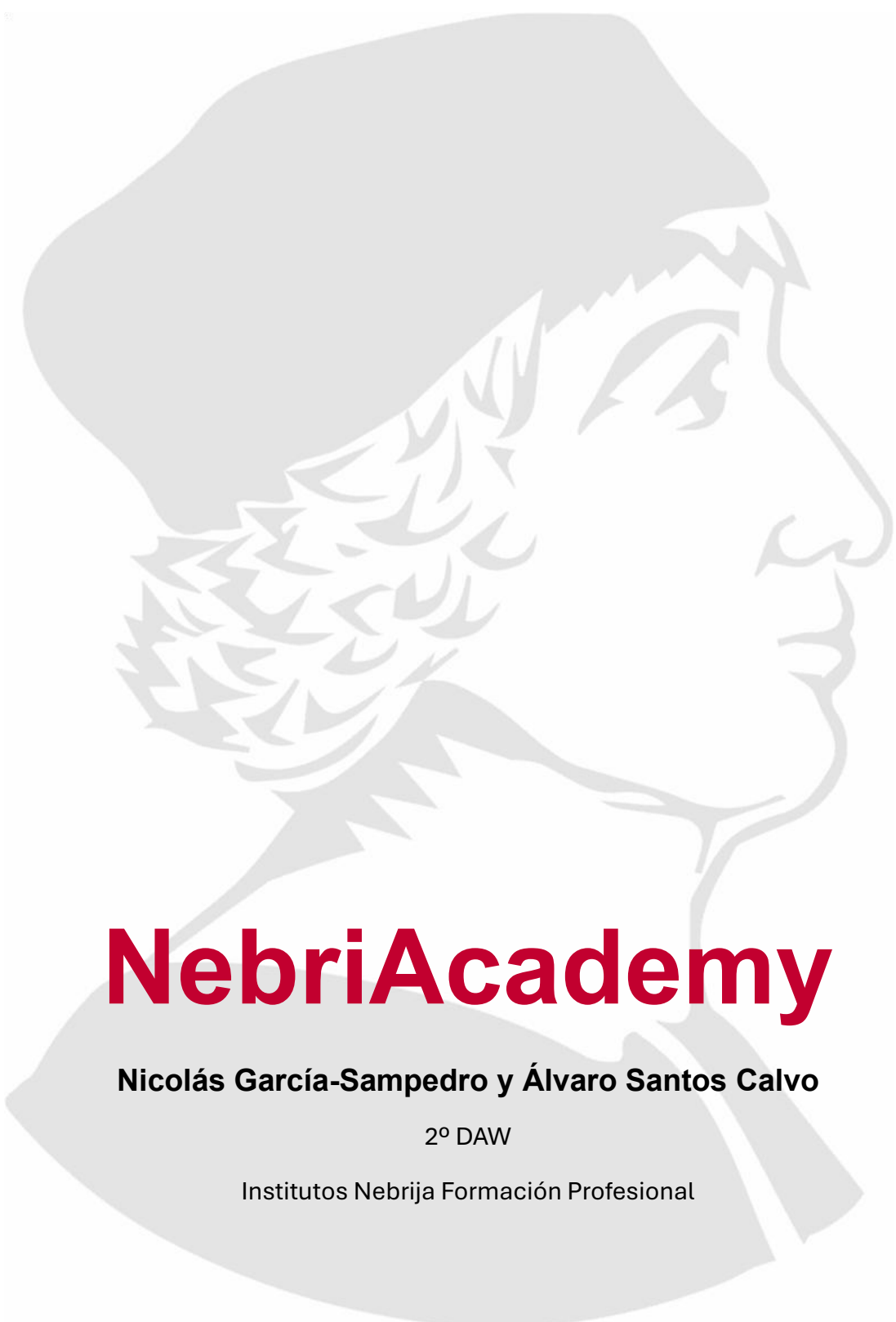




NebriAcademy

Nicolás García-Sampedro y Álvaro Santos Calvo

2º DAW

Institutos Nebrija Formación Profesional

[Repositorio del proyecto](#)

[Página web NebriAcademy](#)

ÍNDICE:

1- AGRADECIMIENTOS:	5
2- INTRODUCCIÓN:	5
Motivación:	6
Objetivos:	6
Temporalización:	8
3- TECNOLOGÍAS USADAS:	8
HERRAMIENTAS DE DESARROLLO Y DOCUMENTACIÓN	8
FRONTEND	10
BACKEND	12
SERVICIOS EN LA NUBE	14
GESTIÓN DE DEPENDENCIAS	15
4- DESCRIPCIÓN DEL PROBLEMA:	16
DAFO:	17
5- ANÁLISIS:	19
Mockups:	19
Diagrama de casos de uso:	20
Documentación de los casos de uso:	22
Diagrama de entidad relación:	33
Diagrama de clases	34
6- IMPLEMENTACIÓN:	35
6.1- Planteamiento del proyecto:	35
6.2- Base de Datos:	36
6.3- Back	37
6.4- Creación del frontend (03/01/26):	39
6.5- Autenticación y Registro (Login y Register):	39
6.6- Navegación (Nav) y Pie de página (Footer)	41
6.7- Home y Mi espacio:	44
6.8- Mi perfil:	47
6.9- Catálogo de cursos:	48
6.10- Curso:	50
6.11- Profesores:	54

6.12- Navegación y seguridad:.....	56
6.13- Apuntes:.....	57
6.14- Soporte y administración:.....	58
6.15- Despliegue:.....	62
6.16- Refactorización y documentación final:	63
7- IMPLEMENTACIONES FUTURAS Y CONCLUSIONES	66
7.1- Implementaciones futuras:	66
7.2- Conclusiones:	68
8- BIBLIOGRAFÍA.....	70
9- ANEXOS:.....	73

1- AGRADECIMIENTOS:

Queremos que quede constancia de nuestro agradecimiento, sobre todo a dos de nuestros profesores del Instituto Nebrija Formación Profesional, por un lado, a Rubén González Martín, que nos ha enseñado todo lo que sabemos sobre el backend y a Francisco Albiar Jiménez, que nos ha transmitido sus conocimientos sobre frontend y nos ha dado todo su apoyo en las tutorías.

Sin la colaboración de nuestros profesores, esta página web no habría sido posible.

2- INTRODUCCIÓN:

Desde nuestra experiencia como estudiantes y alumnos, hemos vivido la transformación de la enseñanza y su evolución desde la impartición en centros de manera presencial hacia el mundo digital. Hemos visto como el *e-learning* ha logrado cambiar la perspectiva de la educación, a través de su evolución a una experiencia totalmente accesible desde cualquier lugar del mundo e incluso utilizando cualquier dispositivo.

Hoy en día, para aprender inglés, ya no tienes que viajar a Londres, sino que desde tu habitación puedes acceder a un sinfín de recursos educativos, de los que puedes disponer en cualquier horario, según tu disponibilidad y recursos.

A su vez, este mayor acceso a la formación por parte de los alumnos ha ocasionado que el mercado laboral exija, a su vez, profesionales tecnológicos más formados, con habilidades actualizadas, lo que supone un aprendizaje continuo, flexible y personalizado.

Tras analizar el mercado, detectamos que muchas de las plataformas formativas existentes (como OpenWebinars, Google Classroom o Moodle) carecen de metodologías verdaderamente interactivas o interfaces intuitivas, así como que no disponen de adaptabilidad a distintos dispositivos como móviles o tablets, lo que dificulta el aprendizaje tal y como se demanda en la actualidad.

Por todo esto, hemos centrado nuestro trabajo de fin de grado en la creación de una plataforma web de e-learning y, en este contexto, hemos creado NebriAcademy,

diseñada para ofrecer una experiencia de usuario intuitiva, fluida, colaborativa y centralizada.

El sistema que hemos desarrollado busca facilitar la gestión académica a todos los niveles, tanto de alumnos, como de profesores, proporcionando un espacio digital orientado a necesidades formativas reales.

Motivación:

Nuestra motivación principal para desarrollar esta plataforma de formación online ha sido superar la rigidez de las soluciones tradicionales, creando un espacio bidireccional que permitiera una interacción real y efectiva. Queríamos ir más allá del consumo pasivo de información, fomentando una comunidad donde los alumnos no solo reciban clases, sino que puedan aportar valor subiendo sus propios apuntes para ayudar a otros compañeros.

Además, como motivación social e institucional, nos propusimos desarrollar una herramienta accesible, estableciendo como pilar fundamental el acceso gratuito a la plataforma para todos los estudiantes de la "familia Nebrija". A nivel administrativo, nos motivó el reto de optimizar la gestión del centro mediante la automatización de la creación de cuentas, la supervisión del rendimiento en tiempo real y la creación de un entorno seguro para el control eficiente de los contenidos multimedia.

Objetivos:

Para hacer realidad NebriAcademy y solventar las carencias detectadas en otras plataformas, nos marcamos los siguientes objetivos técnicos y funcionales:

- **Desarrollar una arquitectura moderna y escalable:**

Construir una aplicación web de página única (SPA) con *React*, sustentada por una *API REST* bajo el patrón *MVC*, y desplegar el sistema en una infraestructura en la nube para garantizar su disponibilidad.

- **Implementar un sistema de roles jerárquico y seguro:**

Diferenciar estrictamente los flujos de navegación y permisos para alumnos (consumo y colaboración comunitaria), profesores (creación de cursos y calificación de ejercicios) y administradores (moderación global y gestión de cuentas institucionales).

- **Diseñar una interfaz intuitiva y adaptable (*responsive*):**

Garantizar una excelente experiencia de usuario, tanto en escritorio, como en dispositivos móviles mediante el diseño de componentes a medida, como carruseles interactivos, notificaciones dinámicas y un buscador centralizado.

- **Gestionar contenidos de forma eficiente:**

Integrar servicios de almacenamiento externos para optimizar la subida, alojamiento y distribución de vídeos, documentos y ejercicios, evitando la sobrecarga del servidor.

- **Proveer un sistema de soporte profesional:**

Asegurar la atención al usuario mediante la integración de herramientas de terceros, canalizando la gestión de incidencias y tickets de soporte directamente desde la interfaz de la plataforma.

Temporalización:

*Anexo 5

	Noviembre				Diciembre				Enero				Febrero				Marzo				Abril				Mayo			
	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4
Planteamiento del proyecto																												
BDD y diagramas iniciales																												
Backend (Node.js / Express)																												
Estructura MVC y Sequelize																												
CRUD completo de tablas																												
Lógica de autenticación y roles																												
Cloudinary y gestión de archivos																												
API Jira e integración admin																												
Frontend (React)																												
Navegación, footer y rutas																												
Login, registro y Zustand																												
Módulo de cursos																												
Profesores y apuntes																												
Home, Mi Espacio y carruseles																												
Panel admin y soporte (Jira UI)																												
Refactorización y UX final																												
Despliegue (Aiven / Render / Vercel)																												
Documentación (memoria TFG)																												

3- TECNOLOGÍAS USADAS:

A lo largo del desarrollo de este proyecto hemos utilizado una variedad de tecnologías, herramientas y servicios que describimos a continuación. Las hemos organizado de forma que se entienda no solo qué es cada una, sino también por qué la elegimos y en qué parte del proyecto la usamos.

HERRAMIENTAS DE DESARROLLO Y DOCUMENTACIÓN

Visual Studio Code

Visual Studio Code es el editor de código que hemos usado durante todo el proyecto. Es gratuito, cuenta con una comunidad enorme y permite instalar extensiones para prácticamente cualquier necesidad. Lo hemos utilizado para escribir todo el código de la aplicación, tanto el frontend como el backend.

GitKraken y GitHub

Para el control de versiones usamos Git a través de GitKraken, una aplicación de escritorio con interfaz gráfica que hace mucho más cómodo trabajar con Git sin tener que memorizar comandos. El repositorio lo alojamos en GitHub. Al ser dos personas trabajando en el mismo proyecto, esto fue fundamental: nos permitió trabajar cada uno en su parte sin sobrescribir el trabajo del otro, consultar qué cambios había hecho cada uno y revertir si algo salía mal. GitKraken nos facilitó especialmente visualizar el historial de commits y gestionar las ramas sin complicaciones.

Jira (Atlassian)

Jira es la herramienta que hemos integrado directamente dentro de NebriAcademy como sistema de soporte al usuario. Cualquier usuario de la plataforma puede abrir un ticket de ayuda desde la propia aplicación, y ese ticket se crea automáticamente en Jira a través de su API REST. Desde la app también pueden consultar sus tickets anteriores, ver su estado y añadir comentarios. Jira no es, por tanto, solo una herramienta interna de desarrollo: forma parte de la funcionalidad visible para el usuario final y es el motor del sistema de soporte de la plataforma.

Figma

Figma es una herramienta de diseño de interfaces que funciona en el navegador y permite crear mockups y prototipos de forma visual e intuitiva. La usamos al inicio del proyecto, antes de escribir ningún código, para diseñar cómo iban a quedar las distintas pantallas de la aplicación. Tener el diseño cerrado de antemano nos ahorró tiempo durante el desarrollo, ya que sabíamos exactamente qué construir en cada componente sin tener que tomar decisiones de diseño sobre la marcha.

Draw.io

Draw.io es una herramienta gratuita para crear diagramas de todo tipo. La usamos para generar los diagramas de la documentación del proyecto: el diagrama de casos de uso,

que muestra cómo interactúan los distintos tipos de usuario con el sistema, y el diagrama entidad-relación de la base de datos, que refleja cómo están estructurados los datos y cómo se relacionan entre sí. Resulta muy cómoda porque funciona directamente en el navegador, guarda los archivos en un formato editable y permite exportar a imagen cuando es necesario.

PlantUML

PlantUML es una herramienta que genera diagramas a partir de texto plano, que integramos en Visual Studio Code mediante su extensión. A diferencia de draw.io, donde los diagramas se construyen arrastrando elementos visualmente, con PlantUML el diagrama se describe con una sintaxis propia y la herramienta genera la imagen de forma automática. Esto tiene la ventaja de que el archivo de texto puede guardarse junto al código en el repositorio y actualizarse fácilmente cuando cambia el modelo. Lo usamos para generar el diagrama de clases de la aplicación, que exportamos en PNG y SVG para incluirlo en la documentación.

FRONTEND

React + Vite

React es la librería de JavaScript que hemos utilizado para construir todo el frontend de NebriAcademy. Es una de las más utilizadas en la industria actualmente y se basa en dividir la interfaz en componentes reutilizables, lo que hace que el código sea más organizado y mantenible. Junto con React usamos Vite, la herramienta que empaqueta y sirve el código durante el desarrollo y genera el build de producción. Elegimos Vite porque arranca significativamente más rápido que otras alternativas y hace que el ciclo de desarrollo sea más ágil, ya que los cambios en el código se reflejan casi al instante en el navegador.

React Router DOM

Cuando se construye una aplicación con React, por defecto no existe ningún sistema de navegación entre páginas. React Router DOM es la librería que añade esa funcionalidad. La usamos para definir todas las rutas de la aplicación mediante el componente `BrowserRouter` y para controlar a qué página accede el usuario según la URL. También implementamos rutas protegidas a través de un componente `ProtectedRoute` propio: si no hay sesión activa, la aplicación redirige automáticamente al login; si el usuario autenticado no tiene el tipo requerido para esa ruta (alumno, profesor o administrador), se le redirige a `/Home`.

Zustand

Zustand es una librería de gestión de estado global para React. Sirve para compartir información entre componentes sin necesidad de pasarla manualmente de uno en uno. En el proyecto la usamos con tres stores diferenciados: `useAuthStore`, que guarda los datos del usuario autenticado y los persiste en `localStorage` mediante el middleware `persist` de Zustand para que sobrevivan a recargas de página; `useModalStore`, que gestiona el modal de confirmación reutilizable de la aplicación convirtiendo su apertura en una promesa que se resuelve con la decisión del usuario; y `useToastStore`, que controla las notificaciones flotantes que aparecen brevemente en pantalla al completar acciones. Elegimos Zustand por su sencillez frente a alternativas como Redux, que para un proyecto de este tamaño resultaría excesivo.

Vercel Analytics

Vercel Analytics es el servicio de métricas que hemos integrado en el frontend a través del paquete `@vercel/analytics/react`. Se inicializa mediante el componente `Analytics` incluido en la raíz de la aplicación y permite registrar visitas y eventos de uso de forma anónima. Al estar desplegado en Vercel, la integración es directa y no requiere configuración adicional.

BACKEND

Node.js + Express

El backend de NebriAcademy, encargado de gestionar toda la lógica de negocio y comunicarse con la base de datos, está construido con Node.js y Express. Node.js permite ejecutar JavaScript en el servidor, fuera del navegador, y Express es el framework que facilita definir las rutas de la API y gestionar las peticiones entrantes. Hemos organizado el código del backend siguiendo una arquitectura por capas: rutas, controladores y modelos. El servidor expone una API REST a la que el frontend llama mediante fetch nativo para obtener o modificar datos. El proyecto utiliza Express 5, la versión más reciente del framework, que entre otras mejoras incorpora manejo nativo de promesas en las rutas.

Sequelize ORM

Sequelize es un ORM (Object-Relational Mapper) que nos permite trabajar con la base de datos desde JavaScript sin escribir consultas SQL directamente. Definimos los modelos de datos como objetos de JavaScript —Alumnos, Profesores, Cursos, Apuntes, Ejercicios, Videos, Notificaciones y otros hasta un total de 15 modelos— y Sequelize traduce las operaciones a SQL de forma automática. Esto hace el código más limpio y reduce la probabilidad de errores en las consultas. Internamente utiliza el driver mysql2 para conectarse a MySQL.

MySQL

MySQL es el sistema de gestión de bases de datos relacional que hemos usado para almacenar toda la información de la aplicación: usuarios, cursos, apuntes, ejercicios, vídeos, calificaciones, notificaciones, etc. Los datos se organizan en tablas relacionadas mediante claves foráneas, lo que encaja de forma natural con la estructura de NebriAcademy: un alumno puede estar matriculado en varios cursos, un ejercicio pertenece a un curso concreto, un apunte tiene autor y categoría, etc. Elegimos un

modelo relacional precisamente porque estas relaciones son centrales en el dominio del proyecto.

Multer

Multer es un middleware de Node.js para recibir archivos en peticiones HTTP multipart/form-data. Sin él, el servidor no sabría cómo procesar una petición que incluye un archivo adjunto. Lo configuramos de dos formas distintas según el caso: con almacenamiento en memoria (memoryStorage) cuando el archivo va a subirse directamente a Cloudinary desde el buffer, y con almacenamiento en disco (diskStorage) en la ruta de Jira, donde el archivo se guarda temporalmente en la carpeta temp_uploads antes de adjuntarlo al ticket.

Streamifier

Streamifier es una utilidad que convierte un buffer de Node.js en un ReadStream. La usamos en el helper cloudinaryHelper.js para poder enviar los archivos recibidos por Multer (que llegan como buffer en memoria) al stream de subida de Cloudinary mediante upload_chunked_stream, sin necesidad de escribirlos primero en disco.

Axios

Axios es una librería para hacer peticiones HTTP desde Node.js. La usamos exclusivamente en el backend para comunicarnos con la API REST de Jira (Atlassian): crear tickets, consultar tickets existentes, añadir comentarios y adjuntar archivos. Es más conveniente que el módulo http nativo de Node porque gestiona automáticamente la serialización del cuerpo, los encabezados de autenticación y la construcción de peticiones multipart junto con form-data.

form-data

form-data es una librería de Node.js que permite construir cuerpos multipart/form-data de forma programática. La usamos en el controlador de Jira para adjuntar archivos a los

tickets, ya que la API de Jira exige que los adjuntos se envíen en ese formato. Se utiliza junto con Axios para enviar los formularios correctamente.

dotenv

dotenv es una librería que carga variables de entorno desde un archivo `.env` local al arrancar el servidor. Las variables de entorno almacenan valores sensibles de configuración (credenciales de base de datos, claves de Cloudinary, tokens de Jira) que no deben escribirse directamente en el código ni subirse al repositorio. Con dotenv esos valores se mantienen fuera del control de versiones en local y se configuran como variables de entorno en los paneles de Render y Vercel en producción.

SERVICIOS EN LA NUBE

Aiven

Aiven es la plataforma en la nube donde tenemos alojada la base de datos MySQL. En lugar de mantener un servidor de base de datos propio, usamos Aiven porque ofrece bases de datos gestionadas con alta disponibilidad y sin necesidad de administración de infraestructura. Esto nos permitió tener la misma base de datos accesible tanto en local como en producción. La conexión requiere SSL y las credenciales se leen siempre desde variables de entorno.

Cloudinary

Cloudinary es el servicio que usamos para almacenar todos los archivos subidos a la plataforma: los PDFs de los apuntes, los vídeos de los cursos y los ficheros entregados en los ejercicios. Guardar estos archivos directamente en el servidor no sería viable en producción por limitaciones de espacio y escalabilidad. Cloudinary recibe el archivo, lo almacena en la nube y devuelve una URL permanente desde la que puede accederse. Cuando se borra un apunte, un vídeo o un ejercicio de la plataforma, el backend también elimina el archivo correspondiente en Cloudinary para evitar acumular recursos huérfanos.

Vercel

Vercel es la plataforma donde hemos desplegado el frontend de la aplicación. Está orientada a aplicaciones React y tiene integración directa con GitHub, lo que permite despliegues automáticos al hacer push al repositorio. Incluye también una configuración de rewrite en `vercel.json` que redirige todas las rutas al `index.html`, necesaria para que el enrutado del lado del cliente funcione correctamente en producción. La URL del backend se configura como variable de entorno `VITE_API_URL` en el panel de Vercel.

Render

El backend lo hemos desplegado en Render, una plataforma orientada a alojar servidores y APIs. Es donde se ejecuta el servidor Express en producción. Todas las credenciales (base de datos, Cloudinary y Jira) se configuran como variables de entorno en el panel de Render, de forma que nunca están expuestas en el código del repositorio.

GESTIÓN DE DEPENDENCIAS

pnpm

pnpm es el gestor de paquetes que hemos usado en este proyecto en lugar de npm. La decisión de cambiar vino motivada por un incidente de seguridad relacionado con paquetes maliciosos publicados en el registro de npm, lo que nos llevó a buscar una alternativa más robusta. Además, pnpm es más eficiente: en lugar de copiar los paquetes en cada proyecto, los almacena en una caché compartida y crea enlaces simbólicos, lo que ahorra espacio en disco y acelera la instalación de dependencias. Tanto el frontend como el backend lo usan para instalar dependencias y ejecutar los scripts de desarrollo y producción.

4- DESCRIPCIÓN DEL PROBLEMA:

Los alumnos y profesores se enfrentan en su día a día a plataformas educativas demasiado rígidas, que no permiten interactuar fácilmente con otros usuarios y que dificultan el aprendizaje colaborativo.

Con la creación de nuestra plataforma de e-learning NebriAcademy, nos proponemos dar solución al problema principal de la falta de un entorno centralizado, dinámico y específico para la comunidad educativa de Nebrija, donde el aprendizaje no sea solo unidireccional (del profesor al alumno), sino que fomente el apoyo mutuo mediante el intercambio libre de documentos, vídeos, ejercicios, apuntes y otros recursos.

Dentro del mercado, existen distintas plataformas de formación (OpenWebinars o Google Classroom), que, si bien están consolidadas, carecen de recursos o funcionalidades que nos parecen imprescindibles en páginas web de aprendizaje on-line.

Plataformas como OpenWebinars no son gratuitas y tienen planes demasiado costosos. En cambio, nuestra plataforma ofrece gratuidad para los estudiantes de la institución, así como planes de pago accesibles al resto de estudiantes.

A su vez, plataformas como Google Classroom suele presentar interfaces poco intuitivas y va dirigida a usuarios prefijados para clases creadas por profesionales concretos.

Hemos creado nuestra plataforma NebriAcademy para cubrir esta carencia, ofreciendo una experiencia accesible para todos los usuarios, resultando, además, gratuita para la comunidad interna y con herramientas específicas que permiten la interacción entre profesores y alumnos mediante calificaciones, apuntes y comentarios.

DAFO:

Análisis DAFO NebriAcademy



- Fortalezas
 - La aplicación funciona en internet real, no solo en el ordenador del desarrollador.
 - Conecta con servicios externos como almacenamiento de archivos y un sistema de tickets de soporte.
 - Tiene tres tipos de usuario bien diferenciados (alumno, profesor y administrador), cada uno con acceso solo a lo que necesita.
 - Al eliminar un curso, se notifica automáticamente a todos los afectados.
 - El código está organizado y bien explicado internamente, lo que facilita que otros lo entiendan o amplíen.
- Debilidades
 - Las contraseñas se guardan sin protección: si alguien accediera a la base de datos, las vería directamente.

- No hay un sistema que impida ataques automatizados, como robots que prueban miles de contraseñas seguidas.
 - El inicio de sesión carga todos los usuarios a la vez, lo que lo haría muy lento con muchos usuarios.
 - La aplicación no comprueba si los datos que recibe son correctos, lo que podría causar errores o ser aprovechado maliciosamente.
 - Los listados muestran todos los registros de golpe, sin páginas, lo que sería un problema con grandes volúmenes de datos.
 - Eliminar contenido es permanente: no hay forma de recuperarlo una vez borrado.
- Oportunidades
 - Con mejoras de seguridad, la aplicación podría acercarse a estándares de productos comerciales reales.
 - La base de datos ya guarda número de tarjeta, por lo que integrar pagos en línea sería una evolución natural.
 - Podría crearse una versión en app móvil reutilizando gran parte del trabajo ya hecho.
 - El buscador de cursos podría mejorarse para ser más rápido y preciso.
 - La estructura modular del proyecto facilita que crezca sin necesidad de rehacerlo desde cero.
- Amenazas
 - Compite con plataformas ya consolidadas como Moodle o openwebinars por lo que debería ofrecer algo específico para el contexto de Nebrija.
 - Depende de servicios gratuitos de terceros que tienen límites: si el uso crece, habría que pagar o cambiar de proveedor.

- No hay pruebas automáticas, así que cualquier cambio futuro en el código podría romper funcionalidades sin que nadie se dé cuenta.
- La falta de protección adecuada de datos sensibles (como DNIs o números de tarjeta) puede incumplir la ley europea de protección de datos (RGPD).

5- ANÁLISIS:

Mockups:

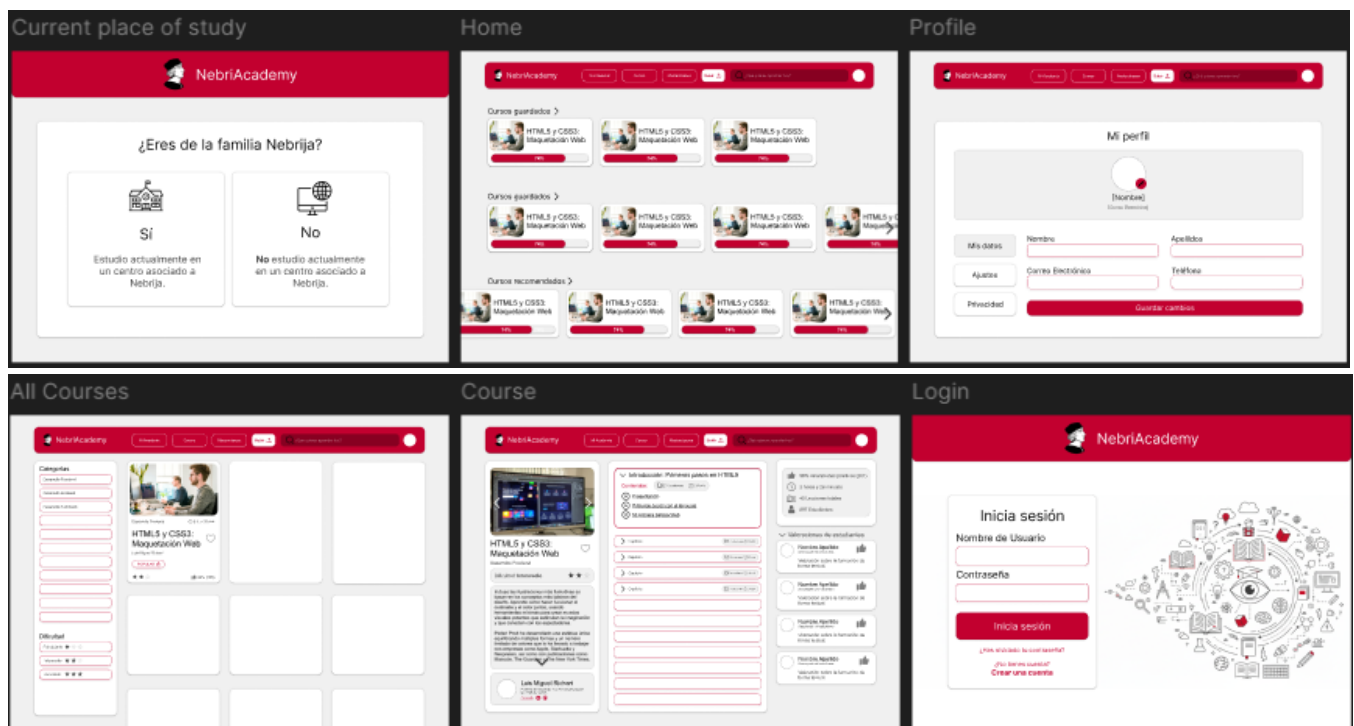
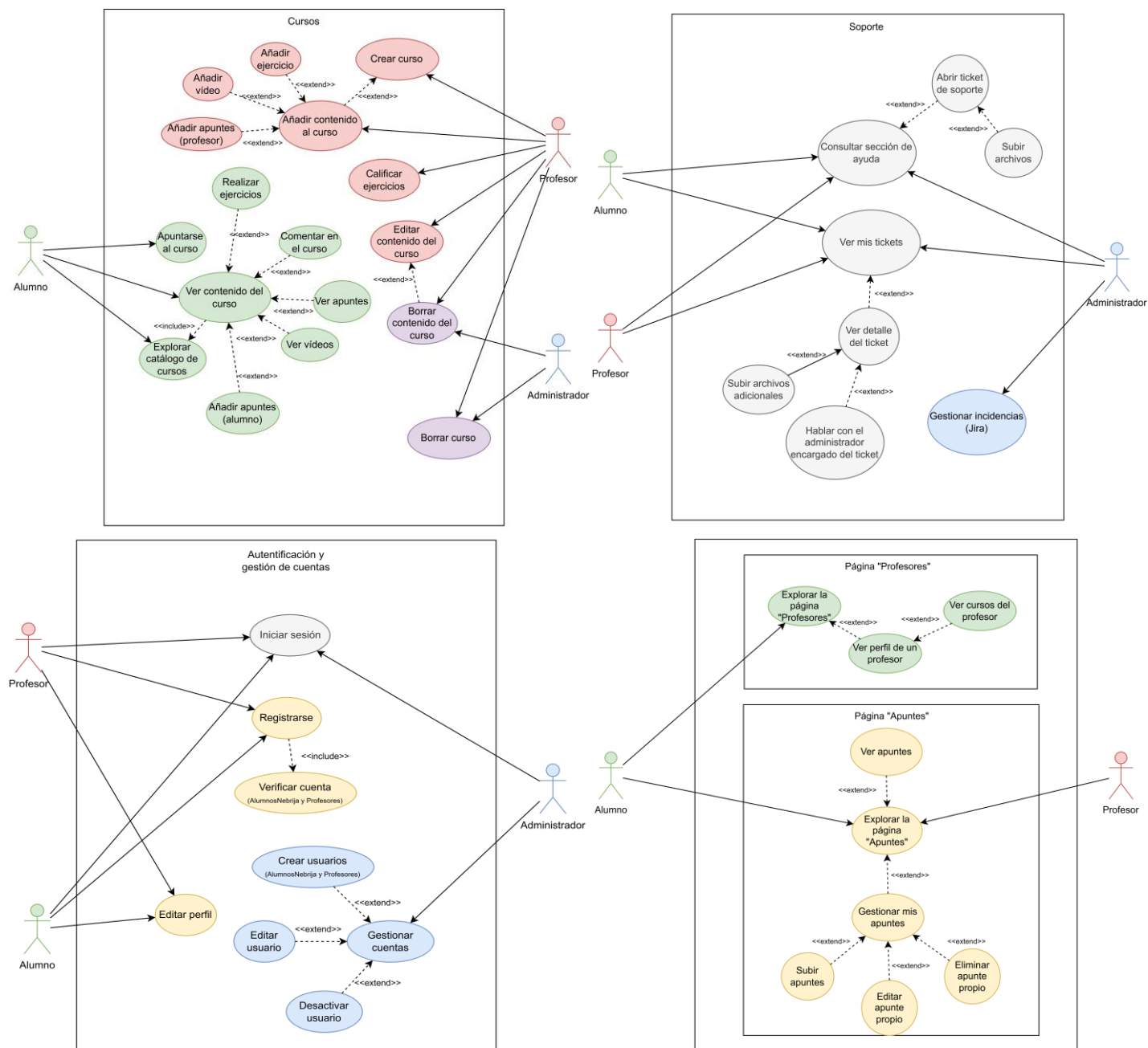
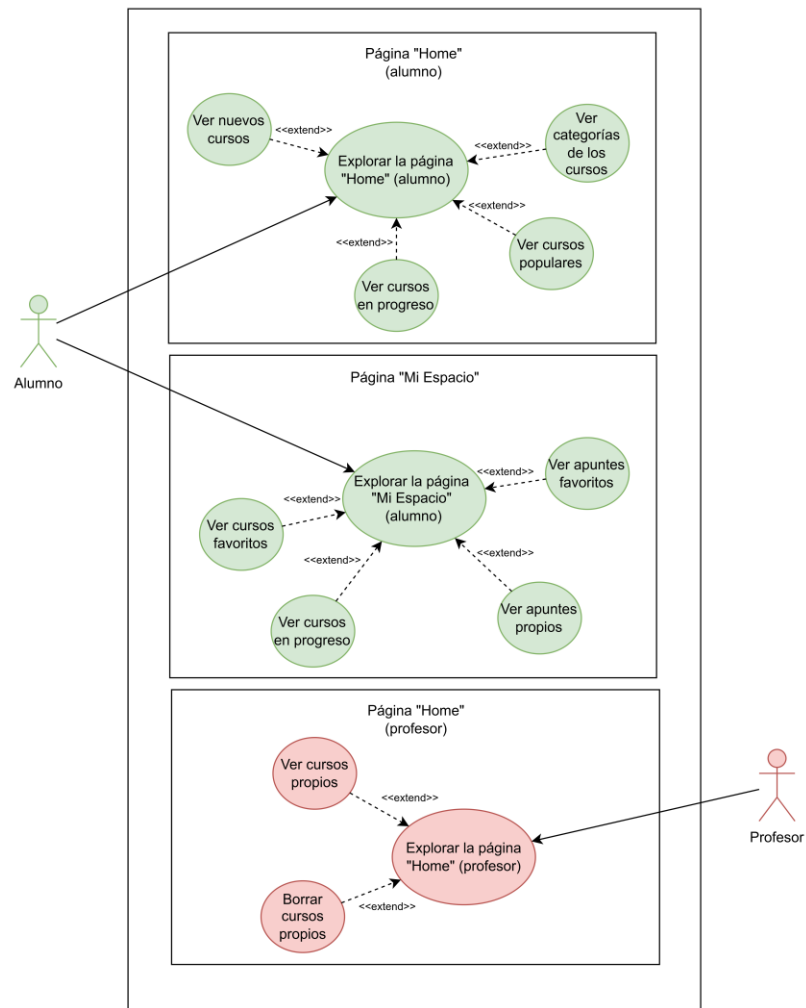


Diagrama de casos de uso:

*Anexo 1





Documentación de los casos de uso:

*Anexo 2

Nombre: Iniciar sesión ID: CU-AUTH-01
Descripción: El usuario accede a su cuenta personal en la plataforma mediante sus credenciales.
Actores: Alumno, Profesor, Administrador.
Precondiciones: El usuario debe estar registrado en el sistema y tener su cuenta verificada.
Curso normal del caso de uso: 1. El sistema muestra un formulario solicitando correo electrónico y contraseña. 2. El usuario introduce sus credenciales y pulsa el botón de entrar. 3. El sistema busca al usuario en la base de datos y valida que la contraseña sea correcta. 4. El sistema concede el acceso y redirige al usuario a su página "Home" correspondiente.
Nombre: Registrarse ID: CU-AUTH-02
Descripción: El usuario crea una cuenta en la plataforma. El proceso varía dependiendo de si es un alumno externo, un alumno de la familia Nebrija o un profesor.
Actores: Alumno, Profesor
Precondiciones: Para los alumnos Nebrija y profesores, el administrador debe haber generado previamente un código de verificación asociado a su correo institucional. Para alumnos externos debe disponer de un correo electrónico no registrado previamente en el sistema.
Curso normal del caso de uso (Ruta de alumnos Nebrija y Profesores): 1. El usuario selecciona la opción de registrarse y el sistema carga la página "PreRegister". 2. El sistema muestra la pregunta "¿Eres estudiante de la familia Nebrija?" con tres opciones: Sí, No y Soy profesor. 3. El usuario pulsa "Sí" o "Soy profesor". 4. El sistema redirige a la página de verificación. 5. El usuario introduce su correo universitario y el código proporcionado presencialmente por la universidad. 6. [Include: Verificar cuenta] El sistema comprueba en la base de datos que el correo existe y que el código es correcto. 7. Al ser correcto, el sistema redirige a la página correspondiente ("Register/alumnonebrija" o "Register/profesor"). 8. El usuario rellena el formulario con sus datos y lo envía. 9. El sistema registra la cuenta en la base de datos.
Flujos alternativos: 3.a. Ruta de Alumno Externo: Si en el paso 2 el usuario pulsa "No". 4.a. El sistema redirige directamente a la página "Register/alumnoexterno (saltándose la verificación)". 5.a. El usuario rellena el formulario y lo envía. 6.a. El sistema registra la cuenta y muestra un mensaje de éxito.

Nombre: Gestionar cuentas ID: CU-AUTH-03
Descripción: El administrador accede a un panel de control para visualizar y administrar a todos los usuarios de la plataforma.
Actores: Administrador
Precondiciones: El administrador debe estar autenticado en el sistema
Curso normal del caso de uso: 1. El administrador entra a la página "Cuentas" desde su barra de navegación. 2. El sistema carga y muestra las tablas con los alumnos y los profesores registrados. 3. Opcionalmente, el administrador decide añadir a alguien al sistema. [Extend: Crear usuarios - Ver ficha CU-AUTH-04]. 4. [Extend: Editar usuario] Opcionalmente, el administrador selecciona una celda de un usuario de la tabla, y la modifica, y el sistema guarda los cambios. 5. [Extend: Desactivar usuario] Opcionalmente, el administrador selecciona un usuario y pulsa en "borrar" El sistema borra la cuenta de la base de datos.
Nombre: Crear usuarios ID: CU-AUTH-04
Descripción: El administrador genera en el sistema las credenciales de acceso (correo institucional y código de verificación) para un alumno o un profesor.
Actores: Administrador
Precondiciones: El administrador debe estar autenticado en el sistema y el usuario físico debe haber solicitado su cuenta presencialmente.
Curso normal del caso de uso: 1. El administrador accede a la página "Cuentas" y selecciona la opción de crear cuenta. 2. El sistema pide un correo y el tipo del usuario (alumno o profesor). 3. El administrador introduce el correo institucional y selecciona el tipo de usuario. 4. El sistema genera un código y guarda la combinación de correo y código en la base de datos, dejándola en estado "Pendiente de registro". 5. El administrador entrega el código de verificación al usuario de forma presencial.
Nombre: Editar perfil ID: CU-AUTH-05
Descripción: El usuario modifica su información personal (contraseña, nº de tarjeta, localidad, etc) dentro de "Mi perfil".
Actores: Alumno, Profesor
Precondiciones: El usuario debe haber iniciado sesión en la plataforma.
Curso normal del caso de uso: 1. El usuario despliega el menú de "Mi perfil" en la barra de navegación y selecciona "Mi perfil". 2. El sistema muestra un formulario con los datos del usuario actuales. 3. El usuario modifica los campos deseados (ej: cambia su contraseña) y pulsa "Guardar Cambios". 4. El sistema valida que los nuevos datos tengan un formato correcto. 5. El sistema actualiza la información en la base de datos y recarga la página mostrando los nuevos datos.

Nombre: Crear curso ID: CU-CUR-01
Descripción: El profesor crea un curso rellenando la información obligatoria y opcionalmente, añade contenido.
Actores: Profesor
Precondiciones: El profesor debe haber iniciado sesión
Curso normal del caso de uso: 1. El profesor pulsa el botón "Añadir curso" en la barra de navegación superior. 2. El sistema muestra el formulario "Crear Curso" con los campos básicos y los paneles opcionales de contenido. 3. El profesor rellena obligatoriamente el nombre, categoría, nivel y descripción del curso, y selecciona una imagen de fondo. 4. [Extend: Añadir contenido al curso] Opcionalmente, el profesor rellena los contenedores de "Añadir Apunte", "Añadir Vídeo" o "Añadir Ejercicio" adjuntando los archivos correspondientes. 5. El profesor pulsa el botón rojo de "Crear curso". 6. El sistema valida que los campos obligatorios estén completos y el formato de los archivos sea correcto (máx. 20 MB). 7. El sistema guarda el curso en la base de datos y redirige al profesor a la página del nuevo curso.
Nombre: Añadir contenido al curso ID: CU-CUR-02
Descripción: El profesor añade nuevo material didáctico (vídeos, apuntes o ejercicios) a un curso que ya ha sido creado previamente.
Actores: Profesor
Precondiciones: El profesor debe estar dentro de la página de uno de sus propios cursos.
Curso normal del caso de uso: 1. El profesor pulsa el botón "+" de opciones dentro del curso. 2. El sistema despliega un menú con las opciones de contenido. 3. [Extend: Añadir vídeo] Opcionalmente, el profesor selecciona "Añadir vídeo", el sistema pide el archivo y título, y lo sube. 4. [Extend: Añadir apuntes] Opcionalmente, el profesor selecciona "Añadir apunte", el sistema pide el archivo (pdf, docx...), descripción y título, y lo sube. 5. [Extend: Añadir ejercicio] Opcionalmente, el profesor selecciona "Añadir ejercicio", el sistema pide el archivo, título y descripción, y lo sube. 6. El sistema actualiza el temario del curso mostrando el nuevo material a los alumnos matriculados.
Nombre: Apuntarse a un curso. ID: CU-CUR-03
Descripción: El alumno accede y se apunta a un curso.
Actores: Alumno
Precondiciones: El alumno debe haber iniciado sesión y entrar a un curso.
Curso normal del caso de uso: 1. El alumno selecciona un curso. 2. El sistema carga la información básica del curso (descripción, nivel, profesor) y muestra los botones de "Favorito" y "Apuntarme". 3. El alumno pulsa el botón de "apuntarme" para acceder al contenido del curso. 4. El sistema registra la matrícula del alumno en la base de datos y desbloquea el contenido.
Flujos alternativos: "Añadir a favoritos": En el paso 3, de forma opcional, el alumno puede pulsar el botón "Favorito". El sistema guardará el curso en la lista de favoritos de su espacio personal sin necesidad de apuntarse.

Nombre: Ver contenido del curso. ID: CU-CUR-04
Descripción: El alumno accede a la página principal de un curso en el que está matriculado para ver el material educativo.
Actores: Alumno
Precondiciones: El alumno debe haber iniciado sesión y haberse apuntado en el curso.
Curso normal del caso de uso: 1. El alumno selecciona un curso. 2. El sistema carga la interfaz del curso y muestra el índice de contenidos disponibles. 3. [Extend: Ver videos] Opcionalmente, el alumno selecciona una lección en vídeo, el sistema carga el reproductor y el alumno visualiza la clase. 4. [Extend: Ver apuntes] Opcionalmente, el alumno hace clic en un documento de teoría, y el sistema muestra el PDF en pantalla. 5. [Extend: Realizar ejercicios] Opcionalmente, el alumno abre una ejercicio, el sistema muestra el PDF, el alumno lo completa y el sistema lo envía para su calificación. 6. [Extend: Comentar en el curso] Opcionalmente, el alumno escribe un comentario o duda en la caja de comentarios de una lección y el sistema lo publica a la vista de la clase. 7. [Extend: Añadir apuntes (alumno)] Opcionalmente, el alumno pulsa el botón "+" para subir nuevo material propio.
Nombre: Calificar ejercicios ID: CU-CUR-05
Descripción: El profesor revisa y puntúa y los ejercicios subidos por los alumnos.
Actores: Profesor
Precondiciones: El profesor debe estar autenticado en el sistema y debe existir al menos una entrega de algún alumno.
Curso normal del caso de uso: 1. El profesor accede a la sección de "Calificar ejercicios" dando al botón de corregir ejercicio, al la derecha del ejercicio. 2. El sistema muestra la lista de ejercicio subidos por los alumnos, indicando el autor de la entrega. 3. El profesor selecciona la entrega de un alumno concreto. 4. El sistema carga el documento enviado por el alumno. 5. El profesor revisa el contenido, introduce la calificación y pulsa "Guardar nota". 6. El sistema registra la nota en la base de datos y se muestra en la interfaz del alumno dentro del curso.

Nombre: Editar contenido del curso ID: CU-CUR-06
Descripción: El profesor modifica los contenidos de un curso ya existente (cambiar títulos, sustituir archivos, etc.).
Actores: Profesor
Precondiciones: El profesor debe haber iniciado sesión y ser el creador del curso que desea modificar.
Curso normal del caso de uso: 1. El profesor entra al curso que desea modificar y pulsa el botón de "Editar contenido del curso" con el símbolo de un lápiz. 2. El sistema añade dos iconos a todos los archivos subidos por el profesor, un lápiz y una papelera. 3. El profesor pulsa el botón del lápiz del contenido que quiere modificar. 4. El sistema muestra un formulario con los datos actuales. 5. El profesor realiza las modificaciones y pulsa "Guardar". 6. El sistema valida que los nuevos datos cumplan con los formatos y restricciones obligatorias. 7. El sistema sobrescribe la información en la base de datos y recarga la página del curso con los datos actualizados.
Flujos alternativos: 3.a. [Extend: Borrar contenido del curso]: Opcionalmente el profesor pulsa el botón de la "papelera" de un archivo del curso. 4.a. El sistema muestra un mensaje de confirmación y un área para que el profesor pueda indicar la razón de la eliminación.
Nombre: Borrar contenido del curso ID: CU-CUR-07
Descripción: Permite eliminar un material específico (un vídeo, apunte o ejercicio) del temario de un curso o comentarios de los alumnos, sin borrar el curso entero.
Actores: Administrador, Profesor
Precondiciones: El usuario debe estar autenticado. El administrador solo necesita acceder al curso; el profesor necesita estar dentro del caso de uso "Editar contenido el curso".
Curso normal del caso de uso (Ruta del Administrador): 1. El administrador accede a la página principal de un curso. 2. El sistema carga el temario mostrando directamente el icono de una "papelera" junto a cada archivo subido, sin necesidad de activar ningún modo de edición. 3. El administrador pulsa el botón de la papelera del contenido que desea borrar. 4. El sistema muestra un mensaje emergente de confirmación y un área de texto para indicar la razón de la eliminación. 5. El administrador escribe el motivo (opcional) y confirma la acción pulsando "Eliminar". 6. El sistema borra el archivo de la base de datos y recarga el temario actualizado sin ese contenido.
Flujos alternativos: • 1.a. Ruta del Profesor (Extensión): El profesor no inicia este caso de uso de forma directa. Accede a él como una extensión opcional al pulsar el botón de la "papelera" mientras se encuentra ejecutando el paso 3 del caso de uso [CU-CUR-06: Editar contenido del curso]. • 2.a. Una vez pulsada la papelera, el proceso salta directamente al paso 7 del curso normal (el sistema muestra recarga la página con los datos actualizados).

Nombre: Borrar curso ID: CU-CUR-08
Descripción: Permite eliminar de forma definitiva un curso completo de la plataforma, incluyendo toda su información, temario y el progreso de los alumnos matriculados.
Actores: Profesor, Administrador
Precondiciones: El usuario debe estar autenticado. Si es profesor, debe ser el creador/propietario del curso.
Curso normal del caso de uso: 1. El usuario Profesor accede a su Home donde aparecen sus cursos y localiza el curso que desea eliminar. Y el administrador accede al catálogo de cursos y localiza el curso que quiera eliminar. 2. El usuario hace clic en el botón u opción de "Borrar curso" (representado con un botón de una "papelera"). 3. El sistema muestra una ventana emergente de alerta crítica ("¿Estás seguro de que quieres borrar este curso? Se eliminará TODO su contenido (videos, apuntes, ejercicios...) y NO se podrá recuperar."). 4. El usuario escribe el motivo (opcional) y confirma la acción pulsando "Confirmar". 5. El sistema borra de forma en **cascada** el registro del curso y sus contenidos en la base de datos. 6. El sistema redirige al usuario al catálogo de cursos (Administrador) o Home (Profesor) mostrando un mensaje de éxito ("Curso eliminado con éxito").
Nombre: Explorar catálogo de cursos ID: CU-CUR-09
Descripción: El alumno navega por la oferta académica de la plataforma utilizando filtros y el buscador para encontrar cursos de su interés.
Actores: Alumno
Precondiciones: El alumno debe haber iniciado sesión en la plataforma.
Curso normal del caso de uso: 1. El alumno hace clic en la sección "Cursos" del navegador. 2. El sistema carga la página general del catálogo y muestra una lista con los cursos. 3. El alumno introduce palabras clave en la barra de búsqueda o selecciona filtros específicos (por ejemplo, filtrar por nivel o por categoría). 4. El sistema filtra los datos en tiempo real y actualiza la lista mostrando solo los cursos que coinciden con los criterios del alumno. 5. El alumno hace clic sobre la tarjeta de un curso concreto para acceder a él.

Nombre: Abrir ticket de soporte ID: CU-SOP-01
Descripción: El usuario accede a la sección de ayuda y envía un reporte (error, denuncia, consulta o sugerencia) al equipo de administración.
Actores: Alumno, Profesor, Administrador
Precondiciones: El usuario debe estar autenticado en la plataforma.
Curso normal del caso de uso: 1. El usuario despliega el menú de perfil (arriba a la derecha) y selecciona "Ayuda". 2. El sistema carga la página de ayuda mostrando el formulario de solicitud. 3. El usuario selecciona el "tipo de reporte" en el desplegable y escribe la descripción del problema o mejora. 4. [Extend: Subir archivos] Opcionalmente, el usuario pulsa "Seleccionar archivo" y adjunta una imagen o documento (máx. 10MB). 5. El usuario pulsa el botón rojo de "Enviar". 6. El sistema guarda la incidencia, la envía automáticamente al tablero de Jira de los administradores y muestra el mensaje de éxito: "Incidencia enviada correctamente".
Nombre: Gestionar incidencias (Jira) ID: CU-SOP-02
Descripción: El administrador recibe, clasifica y resuelve los tickets enviados por los usuarios utilizando el entorno integrado de Jira.
Actores: Administrador
Precondiciones: El administrador debe tener acceso al tablero del proyecto en Jira.
Curso normal del caso de uso: 1. El administrador accede al tablero de incidencias en Jira. 2. El sistema (Jira) muestra los nuevos tickets recibidos con toda la información enviada desde NebriAcademy (usuario, correo, ID, tipo de reporte, descripción y archivos). 3. El administrador se asigna el ticket a sí mismo para indicar que se está encargando de él. 4. El administrador actualiza la "Prioridad" y el "Estado" del ticket (Por hacer, En curso, Esperando al cliente, Resuelto). 5. El administrador utiliza la caja de comentarios (chat) para escribir una respuesta al usuario y guarda los cambios. 6. El sistema sincroniza el nuevo estado y la respuesta para que el usuario pueda verlo en su panel de "Mis Tickets" en NebriAcademy.

Nombre: Ver mis tickets ID: CU-SOP-03
Descripción: El usuario accede a un historial con todos los reportes que ha enviado al equipo de soporte para comprobar su estado general.
Actores: Alumno, Profesor, Administrador
Precondiciones: El usuario debe haber iniciado sesión.
Curso normal del caso de uso: 1. El usuario despliega el menú de perfil y selecciona "Mis Tickets". 2. El sistema consulta la base de datos y muestra una tabla listando todos los tickets del usuario. 3. El sistema muestra para cada ticket: el asunto (tipo de reporte y emisor), el estado actual con su código de color, la fecha de creación y la fecha de última actualización. 4. [Extend: Ver detalle del ticket] Opcionalmente, el usuario pulsa sobre la fila de uno de los tickets de la tabla para acceder a la vista detallada de esa incidencia.
Nombre: Ver detalle del ticket ID: CU-SOP-04
Descripción: El usuario entra a un ticket específico para revisar la conversación con el administrador, leer las respuestas o adjuntar nueva información.
Actores: Alumno, Profesor, Administrador
Precondiciones: El usuario debe estar dentro de "Mis Tickets" y seleccionar una incidencia.
Curso normal del caso de uso: 1. El usuario hace clic en un ticket desde la tabla principal. 2. El sistema carga la interfaz detallada del ticket dividida en dos secciones: información/archivos a la izquierda y el chat a la derecha. 3. [Extend: Hablar con el administrador encargado] Opcionalmente, el usuario escribe un texto en el cuadro "Responder" y pulsa "Enviar comentario". El sistema añade el mensaje al hilo del chat. 4. [Extend: Subir archivos adicionales] Opcionalmente, el usuario selecciona nuevos archivos en el apartado "Adjuntar nuevos archivos" y pulsa "Subir archivos". El sistema los añade al ticket. 5. El usuario pulsa el botón "Volver a Mis Tickets" para regresar a la tabla principal.

Nombre: Explorar la página "Profesores" ID: CU-PROF-01
Descripción: El alumno accede al directorio general de docentes de la plataforma y utiliza herramientas de búsqueda para encontrar profesores según su especialidad.
Actores: Alumno
Precondiciones: El alumno debe haber iniciado sesión.
Curso normal del caso de uso: 1. El alumno hace clic en la pestaña "Profesores" de la barra de navegación superior. 2. El sistema carga la página mostrando un panel lateral de filtros y un contenedor principal con las tarjetas de todos los profesores (imagen, nombre y especialización). 3. El alumno introduce un nombre en el buscador o selecciona un filtro del menú "Especialización" (ej. Ciberseguridad, Programación). 4. El sistema actualiza dinámicamente las tarjetas del contenedor principal mostrando solo los profesores que coinciden con los criterios de búsqueda. 5. [Extend: Ver perfil de un profesor] Opcionalmente, el alumno hace clic sobre la tarjeta de uno de los profesores para ver su información detallada.
Nombre: Ver perfil de un profesor ID: CU-PROF-02
Descripción: El alumno consulta la información detallada de un docente y visualiza el catálogo de cursos que este imparte.
Actores: Alumno
Precondiciones: El alumno debe estar en la página "Profesores".
Curso normal del caso de uso: 1. El alumno hace clic en la tarjeta de un profesor. 2. El sistema carga la página de perfil del profesor seleccionado. 3. El sistema muestra a la derecha la información personal (foto, nombre, correo, país, localización, especialidades) y a la izquierda las tarjetas de los cursos creados por dicho profesor. 4. [Extend: Ver cursos del profesor] Opcionalmente, el alumno hace clic en la tarjeta de uno de los cursos mostrados y el sistema le redirige a la página principal de dicho curso.

Nombre: Explorar la página "Apuntes" ID: CU-APU-01
Descripción: El usuario navega por el repositorio global de apuntes independientes subidos por la comunidad, pudiendo buscar, filtrar y valorar el contenido.
Actores: Alumno, Profesor
Precondiciones: El usuario debe haber iniciado sesión.
Curso normal del caso de uso: 1. El usuario hace clic en la pestaña "Apuntes" de la barra de navegación. 2. El sistema carga la interfaz principal con el buscador, los filtros laterales (Categorías, Mis apuntes, Populares, Novedades) y la lista central de apuntes. 3. El usuario interactúa con los filtros o el buscador. 4. El sistema muestra los apuntes filtrados, indicando en cada tarjeta el nombre, descripción, autor, categoría y número de valoraciones ("likes"). 5. [Extend: Ver apuntes] Opcionalmente, el usuario interactúa con una tarjeta para leer/descargar el apunte o pulsa el botón de "like" para valorarlo positivamente. 6. [Extend: Gestionar mis apuntes] Opcionalmente, el usuario decide interactuar con los botones flotantes inferiores (Lápiz o "+") para administrar su propio contenido.
Nombre: Gestionar mis apuntes ID: CU-APU-02
Descripción: El usuario sube nuevos apuntes libres (no vinculados a un curso) a la plataforma, o edita/elimina los que ya ha publicado anteriormente.
Actores: Alumno, Profesor
Precondiciones: El usuario debe encontrarse en la página "Apuntes".
Curso normal del caso de uso: 1. El usuario pulsa el botón flotante "+" (Añadir). 2. El sistema muestra el formulario "Añadir apuntes". 3. El usuario rellena los campos (Nombre, Descripción, selecciona una Categoría libre) y adjunta el archivo (máx. 20MB). 4. El usuario pulsa el botón rojo "Subir". 5. El sistema valida los datos, guarda el apunte en la base de datos global y recarga la página mostrando el nuevo apunte en la lista.
Flujos alternativos: • 1.a. Activar edición: El usuario hace clic en el botón flotante con forma de "Lápiz" (puede filtrar por "Mis apuntes" para facilitar la edición o eliminación). • 2.a. El sistema revela los botones "Editar" y "Borrar" debajo de las tarjetas de los apuntes que pertenecen a dicho usuario. • 3.a. [Extend: Editar apunte propio] El usuario pulsa "Editar", el sistema carga el formulario con los datos actuales, el usuario los modifica, pulsa "Guardar" y el sistema actualiza la base de datos. • 3.b. [Extend: Eliminar apunte propio] El usuario pulsa "Borrar", el sistema pide confirmación y, tras aceptar, el sistema elimina el apunte de la base de datos y de la vista pública.

Nombre: Explorar la página "Home" (alumno) ID: CU-HOM-01
Descripción: El alumno accede a la página principal de bienvenida de la plataforma para descubrir contenido nuevo, continuar su aprendizaje y explorar categorías.
Actores: Alumno
Precondiciones: El alumno debe haber iniciado sesión en el sistema.
Curso normal del caso de uso: 1. El alumno hace clic en el logo de "NebriAcademy" en la barra de navegación (o el sistema lo redirige aquí tras iniciar sesión). 2. El sistema carga la interfaz principal "Home" con las 4 secciones estructurales. 3. [Extend: Ver nuevos cursos] Opcionalmente, el alumno navega por el slider de "Novedades" e interactúa con las tarjetas de los cursos recién publicados. 4. [Extend: Ver cursos en progreso] Opcionalmente, el alumno navega por el slider de "Tus cursos" y pulsa sobre uno para retomar su estudio. 5. [Extend: Ver categorías de los cursos] Opcionalmente, el alumno pulsa sobre un botón de categoría (ej. Programación) y el sistema lo redirige a la página del catálogo con ese filtro ya aplicado. 6. [Extend: Ver cursos populares] Opcionalmente, el alumno explora el slider inferior para ver los cursos con más valoraciones positivas de la comunidad.

Nombre: Explorar la página "Home" (profesor) ID: CU-HOM-02
Descripción: El profesor accede a su panel de control principal, donde visualiza un resumen visual de todos los cursos que imparte y tiene acceso rápido a las herramientas de eliminación.
Actores: Profesor
Precondiciones: El profesor debe haber iniciado sesión en el sistema.
Curso normal del caso de uso: 1. El profesor hace clic en el logo de "NebriAcademy" o inicia sesión en la plataforma. 2. El sistema carga la vista principal del profesor, mostrando las tarjetas bajo el título "Tus cursos" y habilitando las opciones de edición/borrado. 3. [Extend: Ver cursos propios] Opcionalmente, el profesor hace clic en la tarjeta de uno de sus cursos y el sistema le redirige a la vista principal del mismo para gestionarlo. 4. [Extend: Borrar cursos propios] Opcionalmente, el profesor pulsa el icono de la "papelera" (ya sea la general o la de una tarjeta específica). Esto lanza el flujo de advertencia y borrado definitivo detallado en la ficha cruzada. [Ver ficha CU-CUR-08: Borrar curso].

Nombre: Explorar la página "Mi Espacio" (alumno) ID: CU-ESP-01
Descripción: El alumno accede a su área privada para consultar, en formato de listas rápidas, los cursos y apuntes con los que interactúa habitualmente o que ha creado él mismo.
Actores: Alumno
Precondiciones: El alumno debe haber iniciado sesión en el sistema.
Curso normal del caso de uso: 1. El alumno hace clic en la opción "Mi espacio" de la barra de navegación superior. 2. El sistema carga la vista personal dividida en 4 secciones de listas/sliders. 3. [Extend: Ver cursos en progreso] Opcionalmente, el alumno visualiza o accede a los cursos en los que está matriculado actualmente desde el slider superior. 4. [Extend: Ver cursos favoritos] Opcionalmente, el alumno navega por el slider de "Cursos favoritos" para revisar los cursos que guardó para el futuro. 5. [Extend: Ver apuntes propios] Opcionalmente, el alumno revisa las tarjetas del slider "Tus apuntes" para consultar el material que él mismo ha subido a la plataforma, pudiendo abrirlo directamente. 6. [Extend: Ver apuntes favoritos] Opcionalmente, el alumno navega por el slider inferior para acceder rápidamente a los recursos de otros usuarios a los que les dio "like".

Diagrama de entidad relación:

*Anexo 3

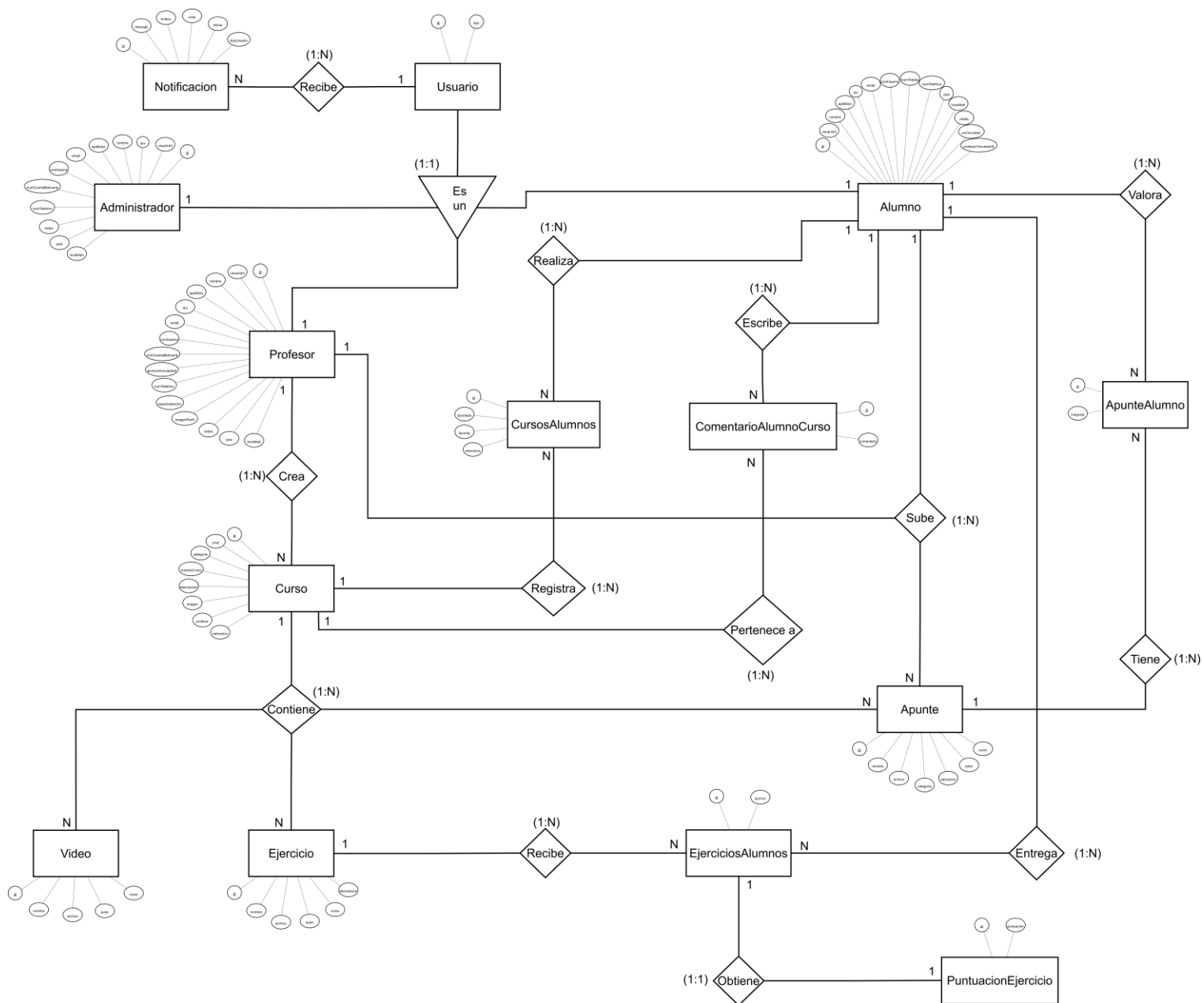
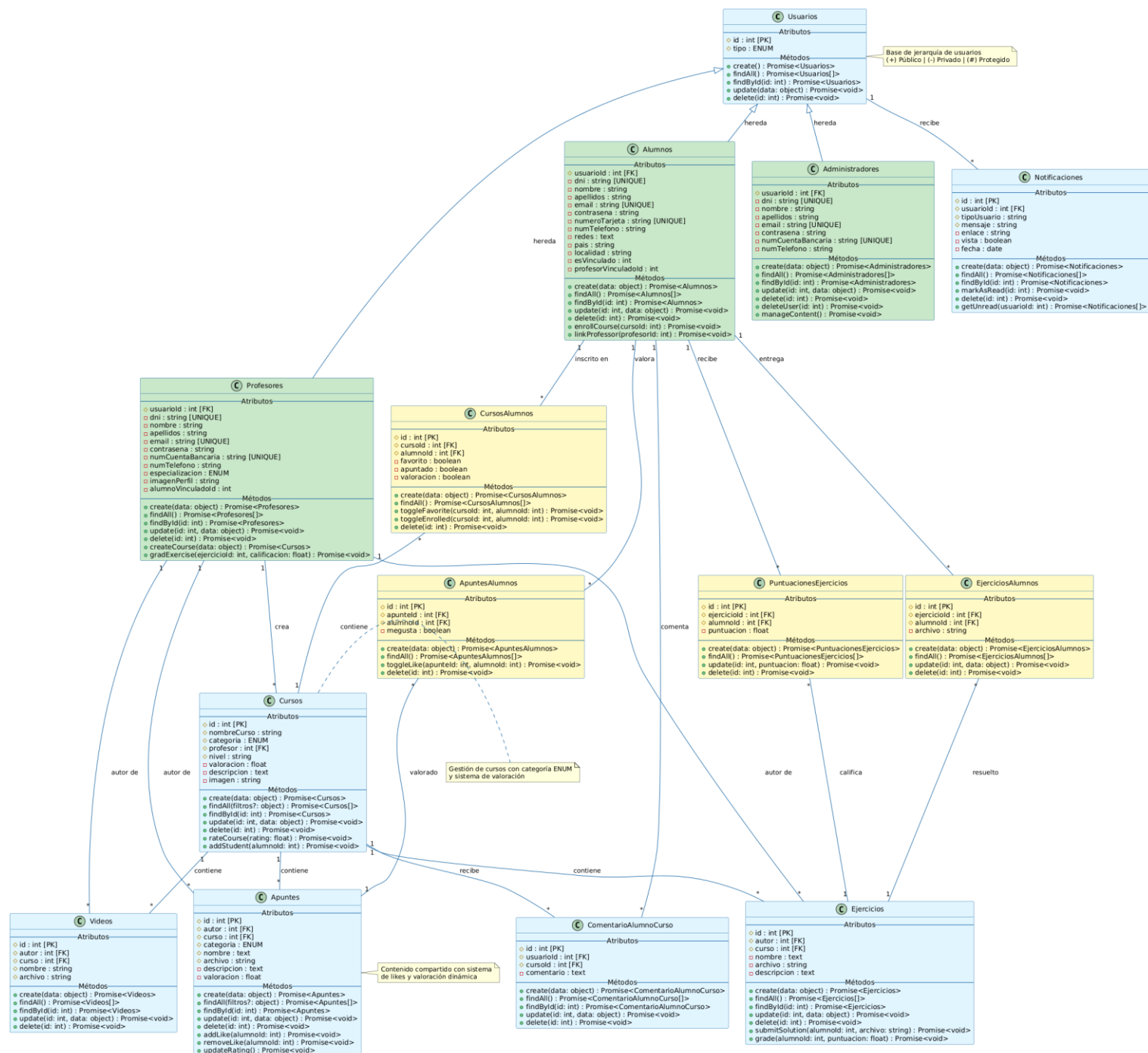


Diagrama de clases:

*Anexo 4



6- IMPLEMENTACIÓN:

6.1- Planteamiento del proyecto:

Antes de iniciar la fase de programación de nuestra página Web, establecimos las reglas de negocio y los flujos de interacción de NebriAcademy. Estructuramos el sistema en torno a tres perfiles de usuario con niveles de autorización estrictamente diferenciados, lo cual sirvió como base para diseñar el modelo Entidad-Relación de la base de datos y configurar los controladores del backend:

- Alumno: Perfil diseñado para el consumo de contenido. Puede tener acceso a los cursos ofertados en la página, cumplimentar una matrícula en aquellos que desee cursar, acceder al contenido completo de los cursos en los que esté registrado (realizar entregas de ejercicios, ver las calificaciones obtenidas) y colaborar activamente con la comunidad, subiendo sus propios apuntes.
- Profesor: Perfil creador. Dispone de permisos para diseñar y gestionar sus propios cursos, publicar material didáctico oficial (vídeos, apuntes y ejercicios) y calificar los ejercicios entregados por los estudiantes.
- Administrador: Perfil de mantenimiento, moderación y soporte. Cuenta con privilegios globales para la creación de las cuentas de los alumnos de Nebrija y profesores, la moderación de cualquier contenido del sistema y la resolución de las incidencias reportadas.

Adicionalmente, se fijó a nivel lógico el requisito de que la plataforma ofreciera acceso gratuito a los estudiantes de la institución, una norma de negocio que posteriormente condicionaría el desarrollo de los flujos de registro e inicio de sesión.

6.2- Base de Datos:

Creación de la BDD (24/11/25):

Para la implementación de la Base de Datos (BDD), utilizamos *XAMPP*, puesto que esta herramienta la habíamos aprendido durante las clases. Nos basamos en los diagramas que habíamos definido previamente para lograr la estructuración de todas las tablas y sus atributos. Posteriormente, incorporamos contenido de ejemplo a estas tablas, con el fin de iniciar el desarrollo de la interfaz de la aplicación.

BDD en la fase de despliegue (09/04/26):

Al iniciar la etapa de despliegue del sitio web, optamos por integrar "*Aiven*", un servicio de alojamiento de bases de datos en la nube, ya que ofrece un plan gratuito y nos resultaba perfecto para lo que necesitábamos, al garantizar disponibilidad ininterrumpida.

A raíz de este cambio, la gestión y conexión con la base de datos se empezó a realizar a través de *MySQL Workbench*, en sustitución de *XAMPP*.

```
// connection.js - SSL obligatorio para conectar con Aiven en la nube
const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    dialect: "mysql",
    dialectOptions: {
      ssl: { rejectUnauthorized: false },
    },
  },
);
```

6.3- Back

Creación del back (25/11/25):

Iniciamos el desarrollo del backend tomando como referencia arquitecturas de proyectos anteriores que habíamos realizado clase. Para organizar la lógica de la aplicación, estructuramos el proyecto utilizando el patrón de diseño *MVC* (Modelo-Vista-Controlador), creando tres directorios fundamentales:

- *Controllers*: para gestionar la lógica de negocio y las respuestas a las peticiones.
- *Models*: para definir la estructura de los datos y la interacción con la base de datos.
- *Routes*: para definir los puntos de acceso de la API y dirigir el tráfico hacia los controladores correspondientes.

Conexión con Sequelize (25/11/25):

Para conectar el back con la base de datos, integramos el *ORM Sequelize*. Esto nos facilitó la creación de tablas, asegurando la información mediante modelos de datos claros.

Centralizamos la configuración y las credenciales en el archivo *conexion.js*.

Al utilizar el driver *mysql2*, delegamos la gestión de las conexiones de bajo nivel al ORM, lo que nos dejó centrarnos en la lógica de negocio y las operaciones CRUD de forma más ágil. Esta infraestructura fue la base clave para desarrollar después la autenticación y la gestión de contenidos.

Enrutado y conexión con BDD (26/11/25):

Primero configuramos el archivo *app.js* como el punto de entrada principal para interceptar las peticiones HTTP.

A través de la carpeta *routes*, definimos los diferentes *endpoints* de nuestra API y dirigimos el tráfico según cada petición.

En el directorio *controllers* centralizamos toda la lógica de negocio, creando las funciones necesarias para ejecutar las futuras operaciones CRUD.

Finalmente, mediante la carpeta *models*, definimos la estructura exacta de nuestros datos usando Sequelize, lo que nos permitió interactuar con la base de datos de forma ágil, segura y sin escribir consultas SQL manuales.

Evolución de la arquitectura del backend y operaciones CRUD (05/12/25 - 22/05/26):

En un principio, con la intención de simplificar el código, decidimos unificar los controladores y las rutas, al considerar que mantenerlos separados no era indispensable (05/12/25).

A partir de esta estructura, intentamos implementar un CRUD básico de prueba, pero no logramos que funcionara correctamente debido a la creciente complejidad del proyecto (07/12/25).

Tras replantear nuestro enfoque y depurar el código, el 13/12/25, conseguimos un gran hito para nuestro proyecto: desarrollar un sistema CRUD completamente funcional para todas las tablas de la base de datos.

Sin embargo, a medida que la aplicación escalaba, nos dimos cuenta de que aquella decisión inicial de unificar rutas y controladores había sido un error, ya que generaba un código difícil de mantener. Por ello, en la fase de refactorización final (22/05/26), reestructuramos por completo las carpetas del backend para separar estrictamente las responsabilidades en capas diferenciadas y cumplir de forma rigurosa con la teoría del patrón MVC.

6.4- Creación del frontend (03/01/26):

Una vez consolidadas las bases del *backend* y la base de datos, iniciamos el desarrollo del *frontend*, utilizando la biblioteca *React*.

Elegimos esta tecnología porque su arquitectura se basa en componentes reutilizables, lo que nos permitía construir una interfaz visual dinámica, escalable y fácil de mantener.

Además, también cuenta con otros beneficios, como facilitar el desarrollo de aplicaciones de una sola página (*SPA*) y contar con una gestión muy eficiente de la interfaz, lo que resulta perfecto para ofrecer a los usuarios una navegación rápida y fluida dentro de la plataforma.

6.5- Autenticación y Registro (Login y Register):

Conexión inicial y lógica de inicio de sesión (19/01/26 - 20/01/26):

Iniciamos este módulo configurando el *middleware CORS* para permitir la comunicación segura entre el *frontend* y el *backend*.

Al desarrollar el inicio de sesión, nos dimos cuenta de que la lógica era compleja, por lo que decidimos extraerla de *routes/usuarios.js* a un nuevo archivo *database/login.js*. De esta forma, nuestro sistema evaluaba las credenciales buscando secuencialmente en tres tablas distintas (administradores, alumnos y profesores) para redirigir al usuario a su panel correspondiente.

Formulario de registro y corrección de transacciones (21/01/26-15/02/26)

Al principio, creamos vistas separadas (mediante un filtro previo) para registrar a cada tipo de usuario: alumnos externos (con información de precios), alumnos de Nebrija y profesores.

Durante las pruebas, detectamos un problema de integridad de datos: si un usuario interrumpía el registro, el sistema guardaba su correo sin contraseña, bloqueando permanentemente la creación de esa cuenta.

Para solucionar esto, tuvimos que cambiar la lógica, de manera que la inserción en la base de datos se realizara solo al enviar el formulario completo.

Flujo de administradores y resolución de fallos (16/02/26-15/05/26):

A medida que el proyecto avanzó, pudimos abordar la implementación de un nuevo tipo de usuario, los administradores, que tenían la función de crear nuevas cuentas para alumnos de Nebrija y profesores.

Esta nueva funcionalidad obligaba al sistema a requerir el dominio `@alumnos.nebrija.es` en la creación de alumnos de Nebrija. El administrador generaba la cuenta con una contraseña temporal, y el usuario accedía a una pantalla de verificación para validarla y establecer la suya propia.

Sin embargo, en este desarrollo nos encontramos con un fallo grave de seguridad: cuando el administrador creaba la cuenta base, el usuario ya podía iniciar sesión sin haber pasado por el proceso de registro. Esto generaba usuarios “fantasma” sin nombre ni apellidos, provocando que cualquier apunte o curso tuviera como autor “Not Null”.

Para solventarlo, modificamos el controlador del *login* para que exigiese la existencia de los datos personales completos, impidiendo el acceso a cuentas a medio configurar.

Además, implementamos una restricción de seguridad para que las cuentas de alumno vinculadas a un profesor (*esVinculado* = 1) no pudieran iniciar sesión de forma directa.

```
// login.js - Validar perfil completo y bloquear cuentas espejo
const alumno = alumnos.find(
  (a) =>
    !a.esVinculado && // bloquea las cuentas espejo de profesores
    a.email === email &&
    a.contrasena === contrasena &&
    a.nombre?.trim() !== "" && // impide acceso a cuentas sin perfil
    a.apellidos?.trim() !== ""
);
```

Refactorización y diseño final (20/02/26-15/05/26):

Finalmente, una vez estabilizada la lógica, aplicamos estilos *CSS responsive* para adaptar las vistas a dispositivos móviles.

Además, llevamos a cabo una refactorización del código, eliminando los archivos individuales de registro y verificación, fusionándolos en componentes comunes para evitar la duplicidad y facilitar su mantenimiento.



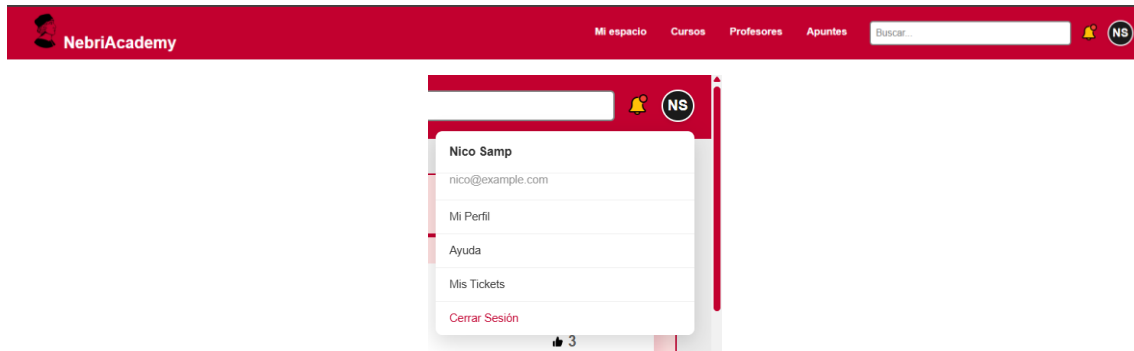
6.6- Navegación (Nav) y Pie de página (Footer)

Estructura base y diseño (21/01/26-30/01/26):

Para el diseño de nuestra página, comenzamos diseñando la barra de navegación (Nav):

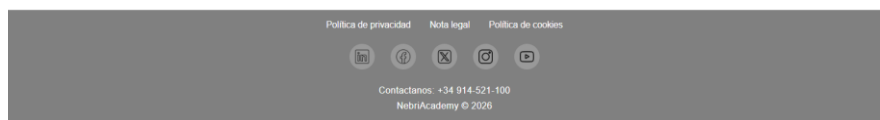
- Utilización del color rojo corporativo de Nebrija .
- Integrando del logotipo de la institución (como enlace al inicio),

- Inclusión de los accesos principales a las secciones de la plataforma.
- Inserción de un menú desplegable de perfil, desde donde el usuario puede consultar su perfil o cerrar sesión.



Para el pie de página (Footer):

- Utilización de un fondo gris que contrastara con la cabecera
- Inclusión de las redes sociales
- Enlaces a las políticas: Las páginas de “Políticas de Privacidad”, “Nota Legal” y “Cookies” se diseñaron con un estilo minimalista para garantizar su legibilidad.

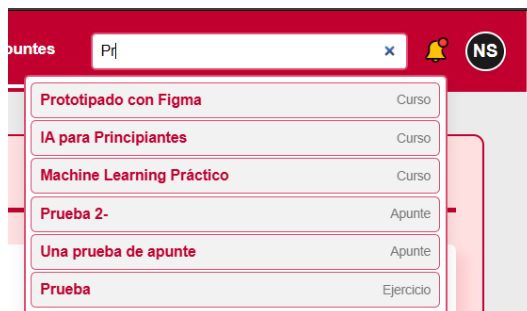


Búsqueda y control de accesos (12/02/26-18/02/26):

Como uno de los objetivos de nuestra página Web era mejorar la experiencia del usuario, en comparación con otras páginas existentes en el mercado, implementamos un buscador centralizado en el Nav que permitía localizar cursos, profesores, apuntes, vídeos y ejercicios.

Para optimizar el rendimiento de este buscador, configuramos la carga inicial mediante *Promise.allSettled*, logrando así una búsqueda instantánea en el *frontend*.

Además, aplicando lógica de negocio en la vista: el buscador se ocultó para los profesores, con el objetivo de evitar que, de forma accidental o maliciosa, pudieran buscar y editar cursos de otros docentes.

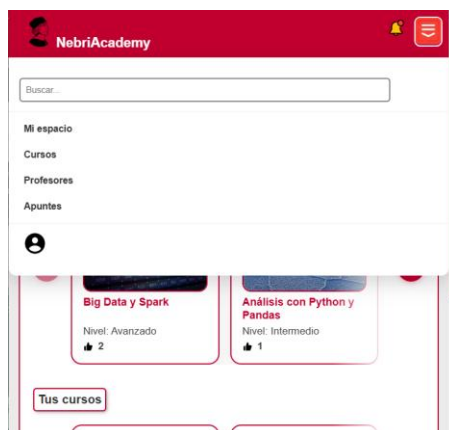


Adaptabilidad móvil y correcciones (13/02/26-18/02/26):

Para pantallas menores a 768px, sustituimos los botones por un menú “hamburguesa”.

Durante las pruebas iniciales, detectamos que, al redimensionar la ventana (por ejemplo, al girar una tablet), la interfaz se rompía, mostrando menús duplicados.

Conseguimos solucionar este problema añadiendo *listeners* que fuerzan el cierre de los desplegables al hacer “clic” fuera o al cambiar el tamaño del *viewport*.



Sistema de notificaciones y unificación del componente (22/04/26-04/05/26):

Inicialmente, habíamos creado dos *Navs* separados: uno para alumnos y otro para profesores.

Sin embargo, en la fase de refactorización, decidimos unificarlos en un único componente dinámico, que mostrara diferentes accesos según el rol del usuario logueado.

También aprovechamos para unificar las tres vistas de políticas en un solo componente reutilizable.

En esta misma etapa, integramos un sistema de notificaciones en tiempo real. Mediante el icono de una campana, el usuario podía desplegar un panel que detallara las interacciones recientes (por ejemplo, si un alumno sube un apunte a un curso, puedes navegar a este archivo simplemente haciendo “clic”, lo que permite limpiar el historial.

Finalmente, refinamos la interfaz general: Simplificamos el CSS de los botones para hacerlos más limpios y utilizamos el *hook* `useLocation()` de *React Router* para marcar visualmente la página activa.

Además, rediseñamos el menú móvil para que ocupara todo el ancho de la pantalla, evitando los redimensionamientos bruscos que provocaba el buscador al mostrar resultados.



6.7- Home y Mi espacio:

Planteamiento y estructura de las vistas (21/01/26-22/04/26):

Antes de diseñar la página de inicio, entendimos que era necesario tener en cuenta y diferenciar las necesidades de cada rol. Y según avanzamos en el proyecto, detectamos nuevas funcionalidades que tuvimos que implementar de cara a mejorar la navegación:

- Para los alumnos, organizamos el *HomeFeed* en cuatro secciones estructuradas mediante tarjetas visuales: “Novedades”, “Tus Cursos”, “Categorías” y “Populares”.
- Para los profesores, creamos una interfaz que permitiera a cada profesor acceder a los cursos de su creación y la opción de gestionarlos, trasladando el botón de creación al Nav para garantizar su accesibilidad.
- Más adelante, desarrollamos *Mi Espacio*, una sección exclusiva para que el estudiante controlara de forma centralizada sus cursos en proceso, favoritos, apuntes creados y apuntes con “me gusta”.
- Con el objetivo de optimizar el código, unificamos el *HomeFeed* de alumno y *HomeProfesor* del profesor en un único componente *Home* mediante renderizado condicional, además de fusionar sus hojas de estilo CSS.

Resolución del problema de los carruseles (Librería vs. Componente Propio) (03/0226-28/04/26):

Al inicio del desarrollo de nuestra página Web, entendíamos que, para mostrar los cursos de forma atractiva, era mejor que estuvieran integradas las librerías *react-slick* y *slick-carousel*.

Sin embargo, detectamos un fallo en su comportamiento *responsive*: al recargar la página desde pantallas pequeñas, las tarjetas no se adaptaban correctamente, encogiéndose y deformando su contenido.

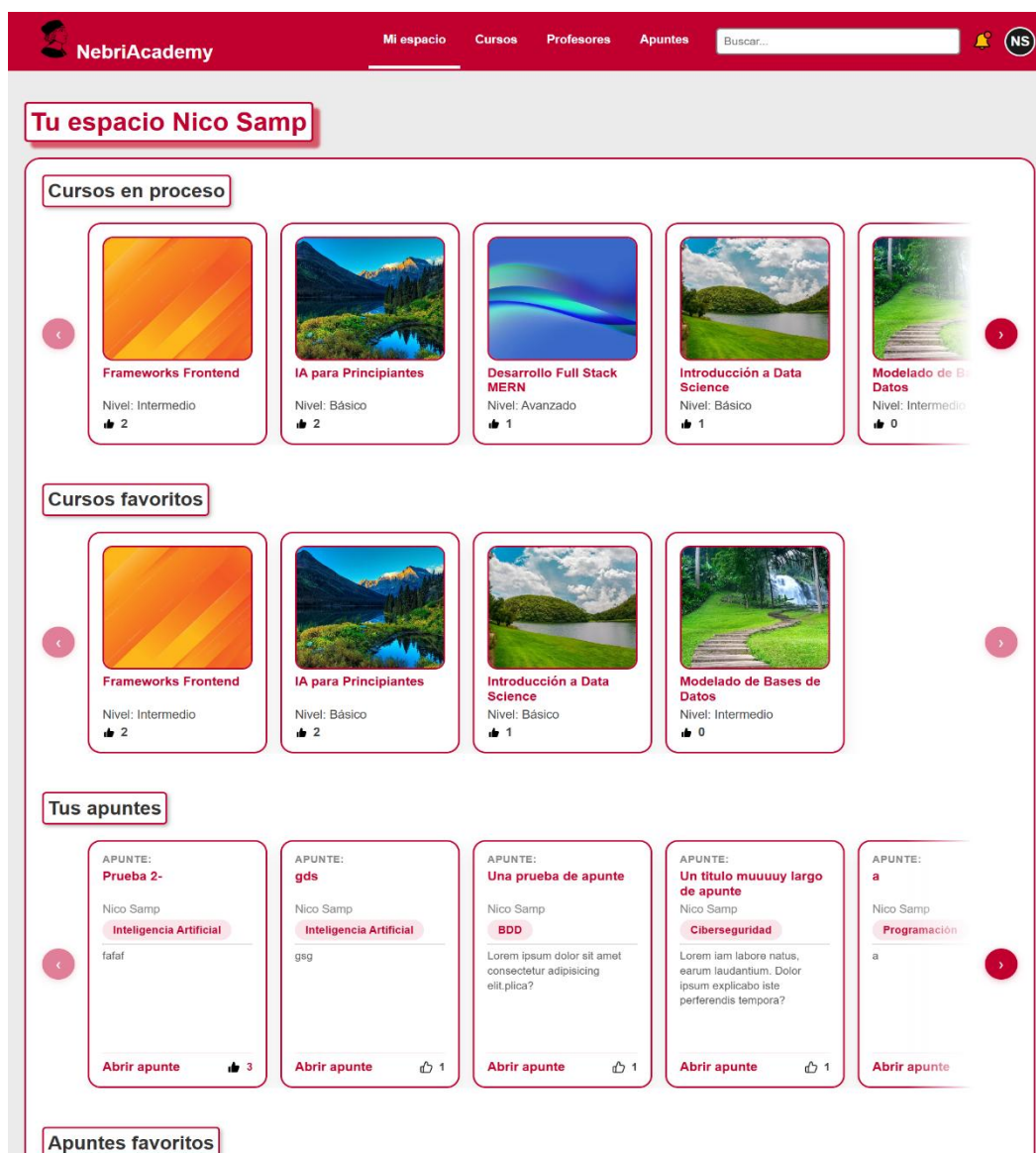
Tras varios intentos fallidos para solucionarlo, utilizando CSS fijo, decidimos eliminar la dependencia externa y desarrollar un componente de carrusel propio (*SliderComponent.jsx*) basado únicamente en *React* y *CSS*.

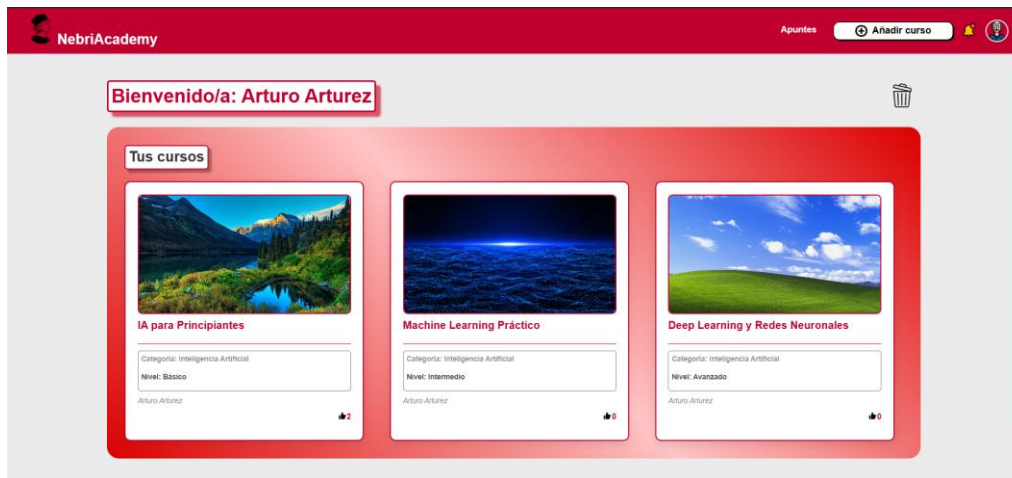
Esta solución resultó completamente fluida y multidispositivo, permitiendo la navegación horizontal, tanto por clics en las flechas, como por deslizamiento táctil.

Unificación de tarjetas y refinamiento visual (28/04/26-30/04/26)

Nos pareció más correcto simplificar al máximo la arquitectura del código, por lo que sustituimos los componentes individuales por una tarjeta común (*CardSlider.jsx*), que adaptaba su contenido dinámicamente, reestructurando el diseño de los apuntes a un formato vertical mucho más accesible.

Finalmente, pulimos la experiencia visual, añadiendo efectos de difuminado en los bordes de los carruseles, para sugerir la existencia de más contenido de forma intuitiva, y configuramos las flechas de navegación para que se desactivasen automáticamente, con un tono más apagado al alcanzar los límites del carrusel.





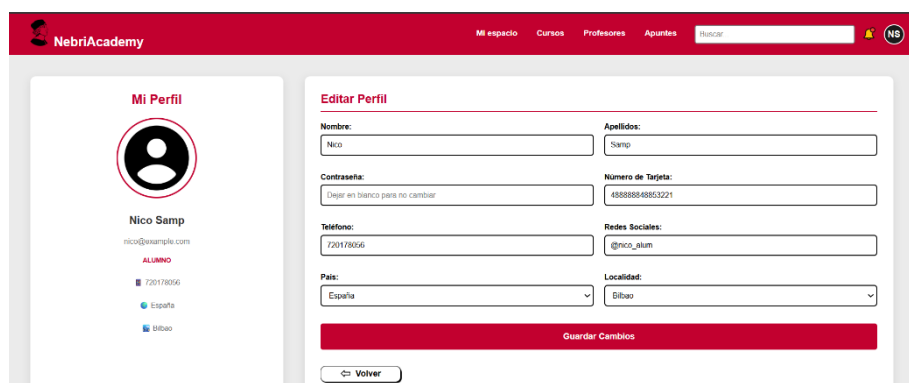
6.8- Mi perfil:

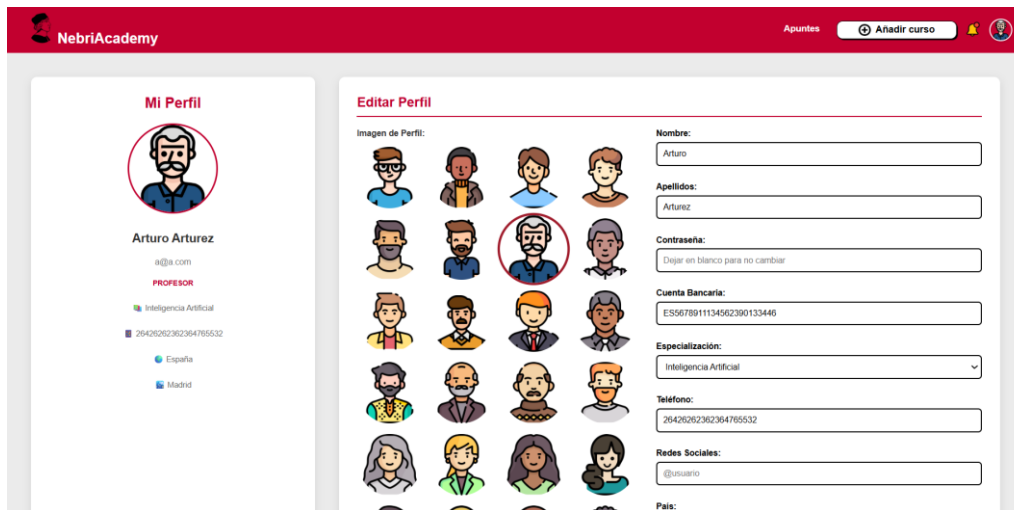
Diseño de la interfaz y persistencia de datos (23/01/26 - 29/01/26):

Diseñamos la sección "Mi Perfil" dividiendo la pantalla en dos bloques proporcionales:

- Un contenedor lateral izquierdo (~30%) para mostrar los datos clave del usuario (nombre, correo, rol y teléfono).
- Uno principal derecho (~70%) con un formulario específico para la edición de sus datos personales.

Al guardar las modificaciones, el sistema actualiza directamente la información en la base de datos. También, incluimos un botón de retorno para facilitar una navegación más cómoda hacia la vista anterior en la que estaba el usuario.





Optimización y unificación de componentes (29/04/26):

Aunque inicialmente desarrollamos esta funcionalidad mediante dos componentes separados (*PerfilProfesorGrid* y *PerfilGrid*), comprobamos que su estructura y comportamiento eran prácticamente idénticos.

Por este motivo, decidimos unificarlos en un único componente dinámico que se adaptara automáticamente según el rol del usuario que haya iniciado sesión. Con esta refactorización conseguimos centralizar la lógica de negocio en un solo punto y eliminar por completo el código duplicado.

6.9- Catálogo de cursos:

Diseño de la interfaz y conexión a datos (23/01/26 - 26/01/26)

Iniciamos el desarrollo del catálogo estructurando la página en dos bloques principales:

- Un panel lateral (*sidebar*) para los filtros
- Un área central con una cuadrícula (*grid*) de tarjetas de cursos. Configuramos las tarjetas para mostrar el nombre, categoría, descripción, profesor, valoración y una imagen de portada.

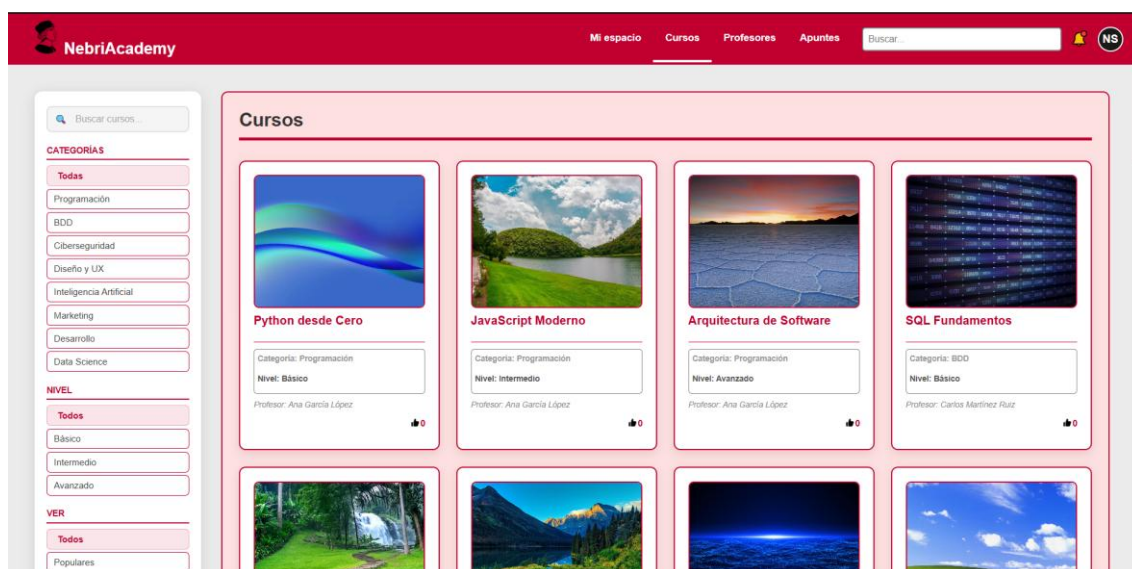
Tras una fase inicial de maquetación con datos estáticos, conectamos la vista con el *backend*, lo que nos permitió que se pudiera cargar dinámicamente el listado de los cursos y la información de sus respectivos profesores.

Para las imágenes, implementamos temporalmente una lógica que asignaba portadas por defecto según el identificador del curso, la cual sustituimos más adelante para que cada profesor pudiera elegir su propia imagen al crear el contenido.

Componente reutilizable de búsqueda (*SearchSidebar*) (04/05/26):

Para evitar la duplicidad de código en las páginas de Cursos, Profesores y Apuntes, decidimos refactorizar los distintos menús de filtrado en un único componente común: *SearchSidebar*.

Este módulo recibe su configuración mediante props, lo que nos permite reutilizarlo inyectando distintos grupos de filtros (como categoría y nivel para cursos, o especialización para profesores) según la vista en la que se integre. Además, mejoramos su adaptabilidad móvil (responsive): en pantallas de escritorio muestra botones interactivos, mientras que en dispositivos móviles se repositiona en la parte superior y transforma las opciones en un menú desplegable (<select>), optimizando el espacio y la usabilidad.



6.10- Curso:

Estructura base y votación (29/01/26):

Diseñamos la vista del curso dividiéndola en dos secciones:

- El contenido principal, en la que se encuentran los vídeos, apuntes y ejercicios de cada curso.
- Los detalles identificativos del curso: profesor que lo imparte, descripción y comentarios al curso efectuados por otros alumnos que lo hayan realizado previamente.

Para el sistema de valoración, en lugar del clásico modelo de estrellas basado en medias, optamos por una lógica de votos positivos/negativos (estilo *Reddit*) que suma o resta la valoración total. Esto permite ver claramente a cuántas personas les ha resultado útil un curso de forma más realista.

Integración de Cloudinary y gestión de archivos (30/01/26-23/04/26):

Inicialmente, utilizamos el *middleware Multer* para almacenar los archivos y vídeos de forma local. Sin embargo, nos dimos cuenta de que esto afectaría drásticamente al rendimiento y almacenamiento del servidor a largo plazo. Por ello, migramos nuestro sistema de almacenamiento a la nube utilizando *Cloudinary*.

Para lograr esto, tuvimos que refactorizar el *backend* para que *Multer* guardara los archivos en la memoria RAM (*memoryStorage*) de forma temporal; un *helper* los subía a *Cloudinary* mediante un *stream* y finalmente sólo almacenábamos la URL pública resultante en nuestra base de datos.

Debido a que usábamos el plan gratuito, tuvimos que aplicar validaciones desde el *frontend*: archivos de texto hasta 10MB y vídeos hasta 100MB.

```
// cloudinary.js - Configuración con credenciales del .env
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key:    process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});
```

```
// apuntes.js (routes) - Multer guarda el archivo en RAM, nunca en disco
const upload = multer({ storage: multer.memoryStorage() });
router.post("/", upload.single("archivo"), controller.create);
```

```
// cloudinaryHelper.js - El buffer de RAM se sube a Cloudinary por stream
function uploadToCloudinary(buffer, options) {
  return new Promise((resolve, reject) => {
    const stream = cloudinary.uploader.upload_chunked_stream(options, (error, result) => {
      if (error) return reject(error);
      resolve(result); // solo guardamos result.secure_url en la BD
    });
    streamifier.createReadStream(buffer).pipe(stream);
  });
}
```

Creación de Cursos, Ejercicios y Apuntes dinámicos (05/02/26-16/02/26):

Desarrollamos una interfaz que permitiera a los profesores añadir cursos cuando le dan al botón de “Añadir Curso”, mediante un formulario seccionado en bloques lógicos (Información general, Apunte, Vídeo, Ejercicio e Imagen de fondo).

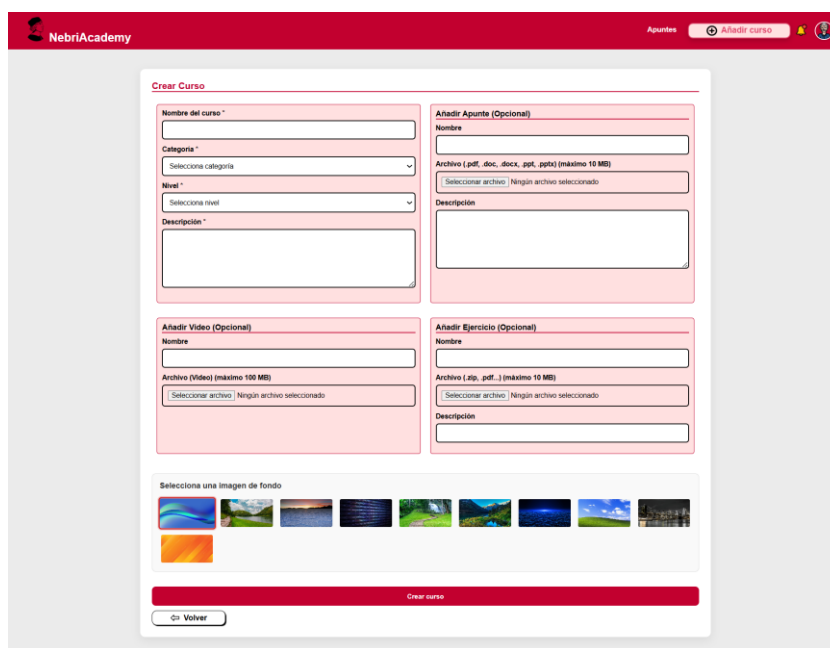
Para los ejercicios, diseñamos un flujo de entrega bidireccional: el alumno sube un archivo y el profesor accede a un panel de corrección (*PuntuarEjercicios*) donde puede evaluar la entrega (del 1 al 10) o rechazarla para un nuevo intento.

Por seguridad, tuvimos que solucionar un error de enrutamiento, puesto que vimos que el sistema permitía a los administradores acceder a esta vista de calificación. Lo

solucionamos restringiendo el acceso de los administradores, para que no pudieran entrar a la vista de calificación de los ejercicios subidos por los alumnos.

Un segundo error detectado fue que los alumnos podían subir contenido a los cursos, como si fueran los profesores, puesto que no habíamos distinguido los accesos para esta funcionalidad (*AddContenidoCurso*). Conseguimos solucionarlo restringiendo los permisos de subida de contenido para que los alumnos sólo pudieran subir ejercicios en los apartados correspondientes del curso, pero nunca contenido a los propios cursos.

Para mejorar la eficiencia del código, unificamos los componentes de visualización y edición (*AddContenidoCurso* con *AddApunteIndividual*), ya que compartían la misma lógica, y automatizamos las categorías en la BDD mediante un *ENUM*, previniendo errores tipográficos al crear cursos.



The screenshot shows the 'Crear Curso' (Create Course) form in the NebriAcademy application. The form is structured as follows:

- Nombre del curso ***: A text input field.
- Categoría ***: A dropdown menu with the text 'Selecciona categoría'.
- Nivel ***: A dropdown menu with the text 'Selecciona nivel'.
- Descripción ***: A large text area.
- Añadir Apunte (Opcional)**: A section containing a 'Nombre' input field, an 'Archivo (pdf, doc, docx, ppt, pptx) (máximo 10 MB)' input field with a file selection button, and a 'Descripción' text area.
- Añadir Video (Opcional)**: A section containing a 'Nombre' input field, an 'Archivo (video) (máximo 100 MB)' input field with a file selection button, and a 'Descripción' text area.
- Añadir Ejercicio (Opcional)**: A section containing a 'Nombre' input field, an 'Archivo (.zip, .pdf...) (máximo 10 MB)' input field with a file selection button, and a 'Descripción' text area.
- Selecciona una imagen de fondo**: A row of small image thumbnails for selection.
- Crear curso**: A red button at the bottom right.
- Volver**: A button at the bottom left.

Sistema de Comentarios (29/01/26-04/05/26):

Nos pareció necesario para mejorar nuestra página, incluir algunas mejoras que la hicieran atractiva para los alumnos, de esta manera, implementamos un sistema de comentarios que permitiera a los alumnos registrados poder debatir dentro del curso. En la vista, el panel de comentarios actualiza dinámicamente el *DOM* sin recargar la

página e incluye una caja de texto con límite de 500 caracteres, ubicada en la parte superior para mayor accesibilidad.

Establecimos un sistema de permisos jerárquicos: los alumnos solo pueden editar/borrar sus propios comentarios, mientras que los profesores pueden borrar comentarios en sus propios cursos y los administradores tienen acceso a los comentarios de todos los cursos, pudiendo editar y borrar los que consideren.

Optimización visual y prevención de concurrencia (30/04/26-15/05/26):

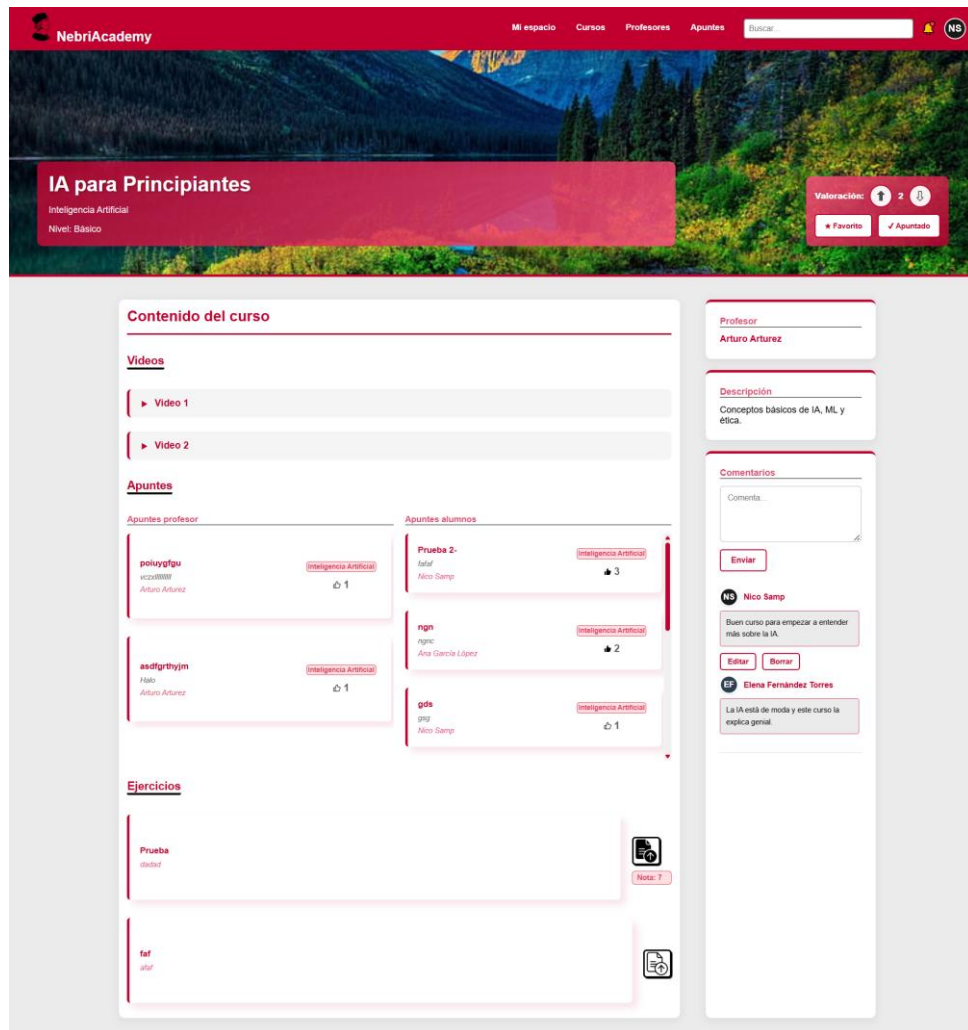
Para facilitar la gestión a los docentes, implementamos un “modo edición” mediante un botón flotante que activara controles directos sobre cada recurso del curso, permitiendo editar o borrar.

En cuanto al *responsive*, configuramos que, a partir de 768px, el panel de detalles se desplazara por encima del contenido y que los comentarios se volvieran colapsables, lo que nos permitía ahorrar espacio en pantalla (también añadimos esta función a los vídeos, haciendo que la página sea mucho más fácil de navegar).

Finalmente, resolvimos un error grave de concurrencia al crear cursos: Detectamos que, si un profesor hacía clic varias veces en “Crear curso”, el servidor registraba múltiples peticiones idénticas, duplicando el curso en la BDD con distintos identificadores. Lo solucionamos implementando una lógica de control en el *frontend* que deshabilitaba el botón de envío durante 8 segundos tras el primer clic, asegurando la integridad de las transacciones.

```
// AddCourseGrid.jsx - useRef para bloquear envíos duplicados sin provocar re-renders
const locksRefLocal = useRef({});

const acquireLockLocal = (key, delay = 800) => {
  if (locksRefLocal.current[key]) return false; // ya hay un envío en curso →
  locksRefLocal.current[key] = true;
  setTimeout(() => delete locksRefLocal.current[key], delay);
  return true;
};
```



6.11- Profesores:

Diseño de las vistas de directorio y perfil (01/02/26 - 02/02/26):

Para diseñar la sección del profesorado, desarrollamos dos vistas principales. En primer lugar, creamos *TodosProfesoresGrid*, un directorio visual compuesto por tarjetas que mostraran la imagen, el nombre y la especialización de cada docente.

Al seleccionar una de estas tarjetas, el usuario podría acceder a *InfoProfesor*, una página de perfil detallada y dividida en dos contenedores:

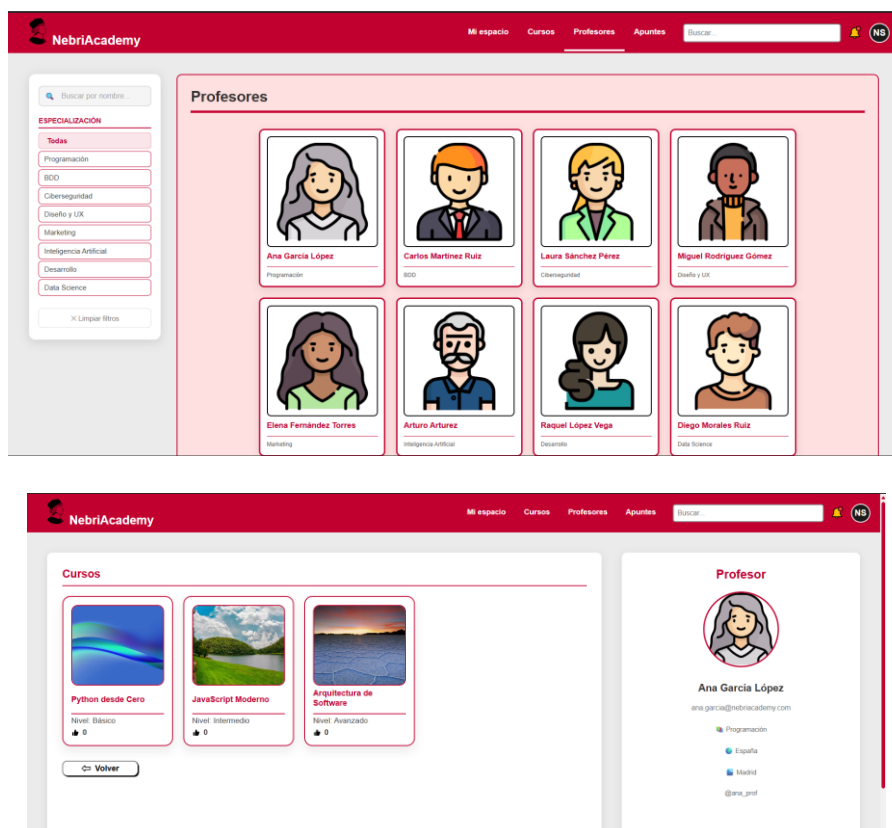
- Uno izquierdo con el listado de los cursos creados por el profesor.
- Uno derecho con sus datos de contacto (correo, país, localidad) y redes sociales.

De cara a asegurar una óptima navegación en dispositivos móviles, aplicamos un diseño *responsive* mediante CSS que apilara automáticamente estos bloques de forma vertical en pantallas reducidas.

Integración del sistema de búsqueda (04/05/26):

Inicialmente, desarrollamos un buscador propio de forma individual para esta vista. Sin embargo, al comprobar que estábamos generando código duplicado con respecto a las páginas "Cursos" y "Apuntes", decidimos diseñar el componente común *SearchSidebar* e integrarlo también en esta página.

Al alimentar este módulo reutilizable con las *props* específicas de los profesores, habilitamos el filtrado por nombre y especialización, lo que nos permitió mantener una interfaz coherente en toda la plataforma y reducir drásticamente la complejidad del código.



6.12- Navegación y seguridad:

Gestión de estado global y rutas protegidas (05/02/26):

Decidimos implementar la librería *Zustand* para gestionar de forma global las sesiones de usuario y asegurar las rutas protegidas de la aplicación, sustituyendo el manejo manual e inseguro en el *localStorage*.

Centralizar los datos de usuario nos permitió restringir el acceso a usuarios no autenticados, mediante un redireccionado automático al formulario de inicio de sesión.

Con esta arquitectura, corregimos un fallo que permitía a los profesores visualizar o editar cursos ajenos, limitando sus permisos exclusivamente a sus propios contenidos.

Además, por seguridad, eliminamos el envío de la contraseña desde el *backend* tras completar la autenticación, para evitar almacenar datos sensibles del cliente.

Control de excepciones y estados de carga (11/02/26 - 04/05/26):

Para mejorar la experiencia de usuario y evitar pantallas en blanco ante rutas inexistentes, creamos una vista personalizada de error 404 (*notFound*) con un botón de retorno integrado.

Adicionalmente, añadimos una animación de carga para las peticiones al servidor: Mientras se estuviera a la espera de la obtención de los datos, el sistema mostraría un *spinner* interactivo con el color corporativo de la universidad.

Unificamos estos estilos en un único archivo CSS compartido, para asegurar la consistencia estética, tanto en las pantallas de espera, como en los avisos de colecciones vacías.

Migración de dependencias a *pnpm* (14/05/26):

Debido a una vulnerabilidad crítica de seguridad reportada en el ecosistema de *npm*, tomamos la decisión estratégica de migrar por completo el gestor de paquetes de nuestro proyecto a *pnpm*.

Esta transición eliminó los riesgos de seguridad gracias a su modelo estricto de resolución de dependencias, el cual impedía el acceso a paquetes indirectos.

Como beneficio secundario, optimizamos el espacio en disco y aceleramos significativamente los tiempos de instalación de los módulos del proyecto.

6.13- Apuntes:

Diseño de la vista y gestión de contenido (08/02/26 - 12/02/26):

Iniciamos el desarrollo de esta sección creando una estructura similar a la del catálogo de cursos y profesores. Diseñamos una vista principal dividida en un panel lateral de filtros y un área central para listar el contenido.

Para fomentar la interacción, incorporamos un botón flotante que permitiera a los usuarios subir material de estudio directamente desde esta vista. Al no estar ligados a un curso específico, estos apuntes de subida libre permitían al usuario asignar la categoría que desee.

Además, mejoramos la usabilidad, modificando el botón de cancelación del formulario para que devolviera al usuario a la vista anterior de forma dinámica.

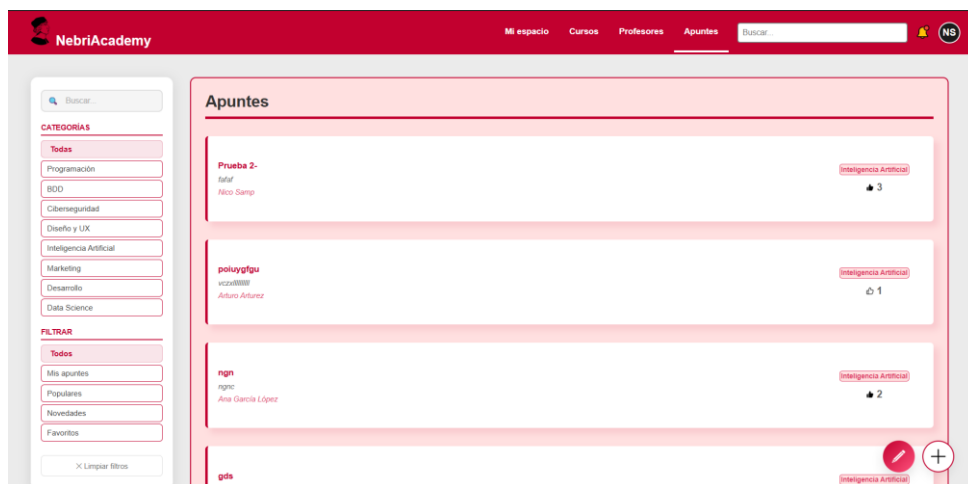
Las tarjetas individuales se diseñaron para mostrar el nombre del archivo como enlace, la descripción, la categoría y un botón interactivo de valoraciones.

Refactorización y unificación de componentes (04/05/26):

Al igual que en otras vistas principales, detectamos código duplicado en la implementación inicial del buscador. Por ello, integramos el componente reutilizable *SearchSidebar*, alimentándolo con las *props* específicas de esta sección, para habilitar el filtrado por nombre, categoría, mis apuntes, popularidad y novedades.

Finalmente, abordamos la refactorización de las tarjetas de apuntes. Teníamos dos componentes (*TarjetaApunte* y *TarjetaApunteCurso*) cuya única diferencia era el tamaño y el contexto de renderizado. Decidimos unificarlos en un solo componente dinámico

que se adaptara automáticamente a su entorno, eliminando código duplicado y centralizando el mantenimiento para futuras modificaciones.



6.14- Soporte y administración:

Gestión del rol de administrador y cuentas (13/03/26 - 29/04/26):

Integramos el rol de administrador con permisos globales, lo que les permitiría moderar y eliminar cualquier contenido de la plataforma.

Desarrollamos un panel de control exclusivo (*CuentasGrid*) para la generación de nuevas cuentas institucionales. Para asegurar la integridad de la base de datos, configuramos *endpoints* que operaran en dos fases (creación del usuario base y posterior asignación del rol), eliminando automáticamente el registro inicial si el proceso falla, para evitar la creación de usuarios "huérfanos".

A nivel visual, estructuramos este panel mediante tablas de datos. Al detectar una alta duplicidad de código entre las tablas de alumnos y profesores, refactorizamos la vista, extrayendo la lógica a un componente reutilizable (*CuentasTable*), el cual recibe su configuración mediante *props*.

Además, incorporamos un buscador global que filtra resultados en tiempo real por múltiples campos (nombre, correo, DNI), mejorando sustancialmente la gestión administrativa.



Nombre	Apellidos	Email	Contraseña	DNI	N° Tarjeta	N° Teléfono	Redes	País	
Juan	Fernández Gómez	juan.fernandez@alumnos.nebrija.es	alumno123	66666666F	4111111111111111	600666777	@juan_alum	España	M
Maria	Rodríguez Díaz	maria.rodriguez@gmail.com	alumno456	77777777G	4222222222222222	600777888	@maria_alum	España	S
Pedro	López Toms	pedro.lopez@alumnos.nebrija.es	alumno789	88888888H	4333333333333333	600888999	@pedro_alum	España	Bi

Integración del sistema de tickets mediante API de Jira (16/03/26 - 23/03/26):

Para el soporte técnico, decidimos prescindir de una tabla local de incidencias y delegar esta gestión en Jira (*Atlassian*). Realizamos la integración a través de su API oficial utilizando autenticación *Basic Auth*, con las credenciales codificadas en *Base64* y protegidas como variables de entorno.

Como todas las peticiones se emitían desde una cuenta de servicio unificada, insertamos los datos del usuario solicitante en la descripción del ticket, mediante el formato estructurado ADF de Jira.

También aplicamos etiquetas identificativas automáticas, lo que nos permitía recuperar el historial de cada usuario empleando consultas *JQL* (*Jira Query Language*) sin necesidad de guardar nada en nuestra base de datos local.

```
// jiraController.js - Basic Auth con credenciales del .env codificadas en Base64
const getJiraHeaders = () => {
  const authId = Buffer.from(`${JIRA_EMAIL}:${JIRA_API_TOKEN}`).toString("base64");
  return { Authorization: `Basic ${authId}`, "Content-Type": "application/json" };
};

// Jira no acepta texto plano: exige formato ADF para las descripciones
const campoParagraph = (etiqueta, valor) => ({
  type: "paragraph",
  content: [
    { type: "text", text: `${etiqueta}: `, marks: [{ type: "strong" }] },
    { type: "text", text: valor || "-" },
  ],
});

// Recuperamos el historial del usuario sin guardar nada en BD local, solo con JQL
const jql = `project = ${JIRA_PROJECT_KEY} AND labels = "uid-${usuarioId}" ORDER BY created DESC`;
```

En el *frontend*, construimos una vista de seguimiento donde los usuarios pueden monitorizar el estado de sus tickets ("en curso", "resuelto", "por hacer" o "esperando al cliente"), adjuntar archivos y comunicarse en un chat de dos columnas directamente con los administradores.

En coherencia con este cambio arquitectónico, renombramos las rutas del *backend* de */incidencias* a */jira*.



The screenshot shows the NebriAcademy website header with navigation links: Mi espacio, Cursos, Profesores, Apuntes, and a search bar. Below the header is a red-bordered box titled "Solicita ayuda al equipo de soporte de Nebriacademy". Inside the box, there is a sub-header "¿Necesitas ayuda con algo?" and a prompt "Escribenos y resolveremos tu problema lo antes posible.".

The form contains the following elements:

- A dropdown menu labeled "Selecciona qué quieres reportar" with a downward arrow.
- A text input field labeled "Describe el problema o mejora".
- A section titled "¿Tienes alguna imagen, video o documento? Adjuntalo para que podamos entender mejor el problema (Máximo 10MB):" with a file selection button labeled "Seleccionar archivo" and a status "Ningún archivo seleccionado".
- A red "Enviar" button at the bottom.


NebriAcademy

Mi espacio
Cursos
Profesores
Apuntes



Mis Tickets

Aquí puedes ver todos los reportes que has enviado al equipo de soporte.

Asunto	Estado	Creado	Última actualización
[Error] Incidencia reportada por Nico Samp	ESPERANDO AL CLIENTE	22/4/2026, 9:42:03	22/4/2026, 10:31:08
[Denuncia] Incidencia reportada por Nico Samp	Por hacer	24/3/2026, 12:40:16	20/4/2026, 9:59:03
[Sugerencia] Incidencia reportada por Nico Samp	Por hacer	23/3/2026, 9:44:48	24/3/2026, 10:20:12
[Error] Incidencia reportada por Nico Samp	RESUELTO	17/3/2026, 19:43:08	17/3/2026, 19:46:05
[Denuncia] Incidencia reportada por Nico Samp	En curso	17/3/2026, 19:35:41	23/3/2026, 9:44:04
[Consulta] Incidencia reportada por Nico Samp	Por hacer	17/3/2026, 10:08:28	17/3/2026, 10:08:29
[Denuncia] Incidencia reportada por Nico Samp	Por hacer	17/3/2026, 10:06:42	17/3/2026, 10:06:43


NebriAcademy

Mi espacio
Cursos
Profesores
Apuntes



[Volver a Mis Tickets](#)

[Sugerencia] Incidencia reportada por Nico Samp

Estado: Por hacer
Creado: 23/3/2026, 9:44:48
Última actualización: 24/3/2026, 10:20:12

Descripción

Usuario: Nico Samp
Correo: nicosamp@nec.com
ID: 9
Tipo de usuario: alumno
Tipo de reporte: Sugerencia
Descripción del usuario: Prueba

Archivos adjuntos

1774256841882-DiagramaCasosDeUsoNebriAcademy.png

Adjuntar nuevos archivos

Chat

Nico Samp 24/3/2026, 10:19:43
wdrisd

Nico Samp 24/3/2026, 10:20:00
Hola

Nico Samp 24/3/2026, 10:20:12
k ta?

Responder

Escribe tu respuesta...

Modo "Versión Alumno" para el profesorado (19/03/26):

Implementamos una funcionalidad para que los docentes pudieran experimentar la plataforma desde la perspectiva de un estudiante, lo que nos pareció podía resultar interesante de cada a la creación de contenido de cada uno de sus cursos.

A nivel de base de datos, generamos cuentas de alumno vinculadas estrictamente a cada profesor existente. Para prevenir brechas de seguridad, bloqueamos el inicio de sesión directo de estas cuentas "espejo"; el profesor solo podría acceder a esta cuenta mediante un botón de intercambio de rol en la navegación, el cual actualizaría dinámicamente el estado global en *Zustand*. Asimismo, configuramos eliminaciones en cascada para que este perfil secundario desapareciera automáticamente si se borra la cuenta principal del docente.

Mejoras de experiencia de usuario y notificaciones (04/05/26):

Para refinar la interacción de las acciones de administración y soporte en toda la plataforma, sustituimos las alertas nativas del navegador por *Toasts* (notificaciones emergentes no intrusivas) y modales de confirmación personalizados.

Además, ampliamos la lógica del sistema de notificaciones: al ejecutar una acción destructiva (como el borrado de un curso o un apunte), el administrador o profesor tiene la opción de detallar una justificación en el *modal*, la cual se envía automáticamente como notificación interna a todos los usuarios afectados.

6.15- Despliegue:

Estrategia de infraestructura distribuida (09/04/26):

Para publicar nuestra aplicación web, inicialmente evaluamos plataformas *full-stack* como *Railway*. Sin embargo, dadas las limitaciones económicas y de recursos de sus planes gratuitos, decidimos descartar un despliegue monolítico y optar por una arquitectura distribuida, utilizando servicios especializados en la nube para cada capa del proyecto.

Para la persistencia de datos, seleccionamos *Aiven*, un servicio en la nube que nos proporcionó un alojamiento estable para nuestra base de datos *MySQL*, pasando a gestionar las conexiones y consultas remotas a través de *MySQL Workbench*.

Para el *backend*, elegimos *Render*. Aunque éramos conscientes de que su plan gratuito entra en estado de hibernación tras un periodo de inactividad, para ahorrar recursos, su capacidad para ejecutar entornos dinámicos (*Node.js*) y su perfecta integración continua con nuestros repositorios de *GitHub* lo convirtieron en la opción óptima para nuestra API.

Finalmente, para el *frontend*, optamos por *Vercel*. Pese a tener ciertas cuotas de uso, su infraestructura está altamente optimizada para servir aplicaciones estáticas (*React*), garantizando un rendimiento excelente y una integración muy sencilla con el resto de los servicios.

Variables de entorno y configuración centralizada:

Para lograr que la aplicación funcionase correctamente tanto en nuestro entorno de desarrollo como en producción, llevamos a cabo una refactorización de la configuración. Eliminamos todas las rutas absolutas escritas directamente en el código del *frontend* (`http://localhost:3000`) y las sustituimos por una constante `API_URL` importada desde un archivo de configuración centralizado.

Paralelamente, adaptamos el *backend* para leer las credenciales de conexión a la base de datos mediante variables de entorno (`.env`), manteniendo los valores de `localhost` como respaldo. Esta abstracción nos permitió desplegar actualizaciones en la nube sin necesidad de modificar el código fuente de las conexiones.

```
// api.js - Una sola constante usada en todo el frontend; sin URLs  
hardcodeadas  
export const API_URL = import.meta.env.VITE_API_URL ||  
"http://localhost:3000";
```

```
// vercel.json - Redirige todas las rutas a index.html para que React  
Router funcione en producción  
{  
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]  
}
```

6.16- Refactorización y documentación final:

Estructura y estandarización del código (05/05/26 - 11/05/26):

Para garantizar la escalabilidad y seguir las buenas prácticas de desarrollo, llevamos a cabo una renovación integral de la arquitectura del proyecto.

Renombramos todos los archivos, clases HTML y recursos gráficos (*assets*) al inglés.

En el *frontend*, establecimos una nueva jerarquía de directorios basada en módulos lógicos: *account* (perfiles), *auth* (acceso), *catalogs* (home, mi espacio, cursos, apuntes y profesores), *common* (utilidades), *layout* (navegación), *management* (gestión de contenidos) y *support* (soporte y políticas). Además, para prevenir conflictos de especificidad y errores de clases duplicadas, eliminamos las importaciones individuales de CSS en cada componente y las centralizamos en el archivo principal *app.jsx*.

Mejoras de Interfaz y Experiencia de Usuario (UX/UI) (05/05/26 - 11/05/26)

Dedicamos una fase a pulir los detalles de navegación e interfaz.

Unificamos componentes repetitivos, como el botón de "volver atrás", consolidando su diseño en una hoja de estilos global (*general.css*) para evitar código innecesario.

Personalizamos las barras de desplazamiento (*scrollbars*) de toda la web con los colores corporativos de Nebrija.

Por otro lado, desarrollamos el componente *ScrollToTop.jsx* para solucionar un problema de navegación en *React*: a partir de ese momento, al cambiar de vista, el sistema forzaría a que la nueva página se cargue siempre desde la parte superior, evitando que el usuario aparezca a mitad de página.

Gestión de eliminaciones y mantenibilidad (06/05/26 - 21/05/26)

Refinamos la interfaz de moderación para hacerla más ágil; por ejemplo, logramos que los administradores visualizaran el botón de la papelera directamente en todos los archivos del curso, sin necesidad de activar previamente un "modo edición".

Vinculado a esto, mejoramos la retroalimentación del sistema: implementamos un área de texto en los modales de borrado para que, cuando un profesor o administrador elimine un contenido, deba justificar el motivo, enviándose automáticamente como notificación a los afectados.

Finalmente, revisamos todo el proyecto para añadir comentarios técnicos que explicaran las funciones complejas, las rutas y los estados globales, facilitando el trabajo a futuros desarrolladores.

Diagramas de arquitectura (20/11/26 - 22/05/26):

La evolución constante del proyecto exigió actualizar los diagramas iniciales.

Mientras que los diagramas de *casos de uso* y *Entidad-Relación* se diseñaron en un principio con *Draw.io*, esta herramienta resultó poco ágil para absorber los continuos cambios del diagrama de clases.

Esto nos obligó a realizar una búsqueda de herramientas alternativas. Inicialmente se evaluó *CodeViz*, una extensión de *Visual Studio Code* (*VS Code*) que integra IA para generar diagramas mediante el análisis automático del código.

No obstante, debido a las limitaciones de su versión gratuita, se optó por *PlantUML*, una extensión de *VS Code* basada en *Diagramas como Código*. Al utilizar un lenguaje de marcado declarativo en lugar de una edición gráfica manual, se logró un diagrama escalable, fácilmente modificable e integrable en el control de versiones (*Git*), garantizando así su sincronización con el desarrollo del software.

7- IMPLEMENTACIONES FUTURAS Y CONCLUSIONES

7.1- Implementaciones futuras:

Durante la realización de este proyecto, hemos identificado algunas funcionalidades que nos resultarían una mejora considerable, pero que no podían ser abordadas de momento, debido, por un lado, a la falta de conocimientos y, por otro a la falta de tiempo.

Relacionamos las necesidades detectadas:

- Realización de correcciones por los profesores a los ejercicios individuales de los alumnos:

En primer lugar, en nuestra página, en la actualidad, los profesores únicamente pueden calificar los ejercicios que los alumnos van subiendo. Creemos que, en un futuro, esta función estaría mejorada, si se permitiera al profesor realizar anotaciones, comentarios o correcciones en el propio ejercicio del alumno y reenviárselo a este con el feedback del profesor, de modo a que el alumno pueda conocer sus fallos y aprender para presentaciones futuras.

En nuestra página, la única solución actual a este problema estaría si cada profesor sube un vídeo explicativo para todos los alumnos. Pero a este vídeo accederían todos los alumnos únicamente por estar registrados en el curso, lo que supondría un inconveniente de cara a la realización de los ejercicios por cada alumno (tendrían acceso a los ejercicios corregidos por el profesor, antes de hacer sus propios ejercicios).

- Métodos de pago a los profesores:

Sería muy interesante poder establecer un sistema de pago a los profesores que tuviera en cuenta, por un lado, la cantidad de cursos subidos a la plataforma, y, por otro, las valoraciones que reporten los alumnos que han cursado estos cursos.

De esta forma, se conseguiría incentivar a los profesores a que crearan más cursos y con mejor calidad, lo que, a su vez, lograría que el contenido de la página mejorara y fuera más competitivo, lo que influiría en la captación de nuevos alumnos.

- Métodos de cobro a los alumnos externos a Nebrija:

En el registro de los cursos, ya hemos implementado un apartado en el que los alumnos cumplimenten los datos relativos al sistema de pago mediante tarjeta de crédito, de manera que estos datos quedarían almacenados en la BDD.

De igual manera, hemos previsto un apartado con información referente al precio de la plataforma de formación.

No obstante, no hemos podido implementar la funcionalidad de un sistema de pago completo. Quedaría pendiente la conexión mediante una plataforma de pago segura, pero no disponemos de los conocimientos para este desarrollo y no hemos podido finalizarlo.

- Obtención de certificaciones para los alumnos:

Por falta de tiempo, no hemos podido implementar en nuestra plataforma, la integración de un sistema de descarga del certificado final que se obtiene en el momento de la finalización de cada uno de los cursos.

- Implementación de un sistema de puntos a los alumnos:

Creemos que sería interesante que nuestra plataforma pudiera disponer de un sistema de puntos que se fuera incrementando, en el perfil de los alumnos, cuando van realizando más cursos, otorgándoles determinados beneficios. De esta forma conseguiríamos elevar la motivación de los alumnos por la realización de los cursos.

- Colaboración de los profesores:

En nuestra plataforma, cada curso únicamente puede ser subido por un solo profesor. En un principio, queríamos que existiera la funcionalidad de una colaboración entre varios profesores en un mismo curso, de manera que cada profesor pudiera aportar sus conocimientos y quedaran integrados en un curso único.

No obstante, desechamos esta idea al resultarnos demasiado compleja en inicio y, en la actualidad, si bien disponemos de los conocimientos para realizarla, no hemos tenido tiempo para su desarrollo.

- Creación de rutas que clasifiquen y contengan distintos cursos según áreas diferenciadas de conocimientos:

Para mejorar la accesibilidad a los conocimientos, pensamos que podría resultar interesante crear unas rutas de acceso que contengan cursos relacionados con materias determinadas y que puedan contener niveles de dificultad diferenciados (por ejemplo: BDD nivel básico, BDD nivel intermedio, BDD avanzado). De esta forma, los alumnos tendrían dentro de la misma ruta todos los conocimientos relacionados con cada materia (programación, ciberseguridad, marketing, etc.

7.2- Conclusiones:

A lo largo del desarrollo de “NebriAcademy”, hemos tenido la oportunidad de aplicar, consolidar y ampliar de forma significativa los conocimientos adquiridos durante el ciclo de Técnico Superior en Desarrollo en Aplicaciones Web.

Este proyecto no solo ha representado un reto académico, sino también una experiencia de aprendizaje integral que nos ha permitido enfrentarnos a cada una de las fases de la creación de una plataforma web de e-learning funcional, realista y profesional que resulta accesible desde internet.

Durante la ejecución del trabajo, nos hemos enfrentado a diversos retos técnicos y organizativos: desde el planteamiento de la idea inicial de negocio, diseño y creación de

la base de datos, desarrollo del backend y el frontend, finalizando con el despliegue de la propia web.

El proyecto queda, además, completamente preparado para una posible evolución hacia un formato más complejo o profesionalizado, lo cual demuestra una planificación coherente, escalable y abierta.

En cuanto a la autoevaluación, consideramos que los objetivos propuestos al inicio han sido cumplidos en su mayoría. El trabajo colaborativo ha sido uno de los grandes aciertos del proyecto, permitiendo una división eficaz de tareas y una sinergia que ha enriquecido cada etapa del proceso. Si bien hemos tenido que ajustar nuestras expectativas en relación con el alcance técnico de algunas fases (como los métodos de pago o la creación de rutas), hemos sabido priorizar para asegurar una página web de formación on-line con las funcionalidades básicas, que permita el desarrollo futuro del proyecto.

A nivel académico y profesional, “NebriAcademy” nos ha permitido entender mejor la lógica interna de una página web, así como el flujo de trabajo entre distintas herramientas y tecnologías (MySQL, Node.js, React, etc.)

Hemos ganado experiencia tanto en la organización de tareas como en la documentación del proceso, generando un proyecto que podría formar parte de un portfolio profesional o ampliarse en un entorno de producción real.

En definitiva, “NebriAcademy” ha sido una experiencia formativa intensa y gratificante. Nos ha permitido unir creatividad y técnica en un entorno colaborativo, y nos ha demostrado que estamos preparados para afrontar nuevos retos dentro del mundo del desarrollo web con compromiso, criterio y entusiasmo.

8- BIBLIOGRAFÍA

Aiven Ltd. (s.f.). *Aiven - Your AI-ready Open-Source Data Platform*. Aiven.

<https://aiven.io/>

Atlassian. (s.f.). *Jira | Project Management for the AI Era*. Atlassian.

<https://www.atlassian.com/software/jira>

Atlassian. (s.f.). *The Jira Cloud platform REST API*. Atlassian Developer.

<https://developer.atlassian.com/cloud/jira/platform/rest/v3/intro/>

axios contributors. (s.f.). *axios | Promise based HTTP client*. Axios. <https://axios->

<http.com/>

Cloudinary Ltd. (s.f.). *Image and Video Upload, Storage, Optimization and CDN*.

Cloudinary. <https://cloudinary.com/>

Cloudinary Ltd. (s.f.). *Node.js SDK*. Cloudinary Docs.

https://cloudinary.com/documentation/node_integration

CodeViz. (s.f.). *CodeViz - Visualize Your Code*. CodeViz. <https://www.codeviz.ai/>

draw.io. (s.f.). *Flowchart Maker & Online Diagram Software*. draw.io.

<https://app.diagrams.net/>

expressjs. (s.f.). *GitHub - expressjs/cors: Node.js CORS middleware*. GitHub.

<https://github.com/expressjs/cors>

expressjs. (s.f.). *GitHub - expressjs/multer: Node.js middleware for handling*

multipart/form-data. GitHub. <https://github.com/expressjs/multer>

Figma, Inc. (s.f.). *Figma: The Collaborative Interface Design Tool*. Figma.

<https://www.figma.com/>

form-data contributors. (s.f.). *GitHub - form-data/form-data: A module to create*

readable multipart/form-data streams. GitHub. <https://github.com/form-data/form-data>

Freepik Company. (s.f.). *Flaticon: Free vector icons and stickers*. Flaticon.

<https://www.flaticon.com/>

gagle. (s.f.). *GitHub - gagle/node-streamifier: Converts a Buffer/String to a readable stream*. GitHub. <https://github.com/gagle/node-streamifier>

GitHub, Inc. (s.f.). *GitHub · Change is constant. GitHub keeps you ahead*. GitHub. <https://github.com/>

GitKraken. (s.f.). *GitKraken Legendary Git Tools*. GitKraken. <https://www.gitkraken.com/>

Meta Open Source. (s.f.). *React*. React. <https://react.dev/>

Meta Open Source. (s.f.). *React Router Official Documentation*. React Router. <https://reactrouter.com/>

Microsoft. (s.f.). *Visual Studio Code - The open-source AI code editor*. Microsoft. <https://code.visualstudio.com/>

midudev [@midudev]. (30 de diciembre de 2024). *Diagramas Interactivos en Visual Studio Code* [Video]. TikTok. <https://www.tiktok.com/@midudev/video/7454215923461836064>

motdotla. (s.f.). *GitHub - motdotla/dotenv: Loads environment variables from .env for nodejs projects*. GitHub. <https://github.com/motdotla/dotenv>

npm, Inc. (s.f.). *npm: The package manager for JavaScript*. npm. <https://www.npmjs.com/>

OpenJS Foundation. (s.f.). *Express - Node.js web application framework*. Express.js. <https://expressjs.com/>

OpenJS Foundation. (s.f.). *Node.js — Run JavaScript Everywhere*. Node.js. <https://nodejs.org/>

OpenJS Foundation. (s.f.). *Node.js API documentation*. Node.js. <https://nodejs.org/docs/latest/api/>

OpenWebinars. (s.f.). *Soluciones de aprendizaje para empresas*. OpenWebinars. <https://openwebinars.net/>

Oracle Corporation. (s.f.). *MySQL*. MySQL. <https://www.mysql.com/>

PlantUML. (s.f.). *Sintaxis diagrama de clases y características*. PlantUML. <https://plantuml.com/es/class-diagram>

pnpm contributors. (s.f.). *Fast, disk space efficient package manager | pnpm*. pnpm.
<https://pnpm.io/>

remy. (s.f.). *GitHub - remy/nodemon: Monitor for any changes in your node.js application and automatically restart the server*. GitHub.
<https://github.com/remy/nodemon>

Render Inc. (s.f.). *Render | The cloud for builders*. Render. <https://render.com/>

Sequelize contributors. (s.f.). *Sequelize*. Sequelize. <https://sequelize.org/>

sidorares. (s.f.). *GitHub - sidorares/node-mysql2: fast mysqljs/mysql compatible mysql driver for node.js*. GitHub. <https://github.com/sidorares/node-mysql2>

Universidad Nebrija. (s.f.). *Nebrija, la Universidad*. Universidad Nebrija.
<https://www.nebrija.com/>

Velasquez, M. (15 de agosto de 2025). *PlantUML paso a paso: crea tu primer diagrama de clases en VS Code* [Vídeo]. YouTube.
https://youtu.be/OEB3Kw40AzU?si=XMo5uwgrfuBCUf_i

Vercel. (s.f.). *Vercel Web Analytics*. Vercel Docs. <https://vercel.com/docs/analytics>

Vercel. (s.f.). *Vercel: Build and deploy the best web experiences with the AI Cloud*. Vercel. <https://vercel.com/>

VoidZero Inc. (s.f.). *Getting Started | Vite*. Vite. <https://vite.dev/guide/>

Zustand contributors. (s.f.). *Poimandres documentation*. Pmndrs.
<https://zustand.docs.pmnd.rs/>

9- ANEXOS:

- 1- [Diagrama de casos de uso](#)
- 2- [Diagrama de casos de uso detallado](#)
- 3- [Diagrama de entidad relación](#)
- 4- [Diagrama de clases](#)
- 5- [Diagrama de Gantt](#)